

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Praca dyplomowa licencjacka

na kierunku Matematyka i analiza danych

Modelowanie wyników meczów piłkarskich z wykorzystaniem metod statystycznych i uczenia maszynowego

Stanisław Pawlak

Numer albumu 341733

promotor
dr Robert Małysz

WARSZAWA 2026

Streszczenie

Tytuł pracy: Modelowanie wyników meczów piłkarskich z wykorzystaniem metod statystycznych i uczenia maszynowego

Słowa kluczowe: uczenie maszynowe, modelowanie statystyczne, prognozowanie wyników meczów, piłka nożna, analiza danych sportowych, ranking Elo, rozkład Poissona, model Dixon-Coles, regresja logistyczna wieloklasowa, MNLogit, XGBoost, lasy losowe, Optuna, strojenie hiperparametrów, Log Loss, Brier Score, ważność cech, współliniowość zmiennych, ekonometria sportu

Celem pracy było porównanie jakości modeli uczenia maszynowego, klasycznych estymatorów statystycznych z modelami bazowymi w zadaniu klasyfikacji trójklasowej wyników meczów piłkarskich, gdzie:

- 0 – wygrana gospodarzy
- 1 – remis
- 2 – wygrana gości

Badanie zostało przeprowadzone na odpowiednio przygotowanym zbiorze meczów piłkarskich z lat 2000–2025 z różnych lig, który pierwotnie zawierał około 230 tysięcy obserwacji.

Preprocessing tak obszernego zbioru wymagał kilku fundamentalnych zmian. Odrzucono kolumnę `MatchTime`, ponieważ zawierała ponad 50% braków danych. Ponadto zdecydowano o wykluczeniu lig niszowych zawierających znaczne ilości braków danych. Następnie konieczne było usunięcie zmiennych pomeczowych w celu eliminacji wycieku danych, a także usunięcie rekordów z brakującą etykietą. W celu ulepszenia zdolności predykcyjnej dodano także cechy pochodne jak np. `EloDifference`.

Kolejnym krokiem był trening modeli regresji Poissona z korektą Dixona-Colesa, wieloklasowej regresji logistycznej, drzewa decyzyjnego, lasu losowego oraz XGBoost. W celu optymalizacji hiperparametrów podzielono przetworzony wstępnie zbiór na część treningową, walidacyjną oraz testową. Konieczne było tu zmodyfikowanie podejścia poprzez traktowanie danych jako szeregu czasowego, ponieważ wyniki kolejnych meczów danej drużyny są od siebie zależne oraz mają określony porządek chronologiczny. Aby umożliwić znalezienie wspomnianych optymalnych wartości hiperparametrów użyto narzędzia `Optuna` [1] z zadaniem minimalizacji `LogLoss`. Do przyspieszenia treningu wykorzystano akcelerację GPU.

Wyniki zaimplementowanych modeli, uzyskane w oparciu o wyselekcjonowany zestaw cech, wykazują w zastosowanych metrykach nieznaczną poprawę w stosunku do modeli bazowych. Ponadto rezultaty modeli uczenia maszynowego nie różniły się bardzo od modeli statystycznych, osiągając podobne wyniki: `Accuracy` ok. 50–51%, `LogLoss` ok. 1,00 i `Brier Score` ok. 0,59–0,60. Analiza macierzy pomyłek wykazała, że wszystkie modele miały trudność z przewidywaniem wyników remisowych, co przełożyło się na lepsze wartości metryk w ligach, gdzie statystycznie mniej meczów kończy się remisem. Najważniejszą cechą okazała się wspomniana cecha pochodna `EloDifference`, co wskazuje, że przy obecnym zestawie cech modele nieliniowe dają jedynie niewielką poprawę względem modeli liniowych. W przypadku klasycznych modeli statystycznych znaczenie tej zmiennej również znalazło odzwierciedlenie w wynikach przez bardzo niską p-wartość i wysoką wartość statystyki z tej cechy. Otrzymane rezultaty potwierdzają wysoką użyteczność zaprezentowanego aparatu matematyczno-statystycznego w ilościowej analizie i modelowaniu wysoce losowych procesów sportowych.

Abstract

Title: Modelling Football Match Outcomes Using Statistical and Machine Learning Methods
Keywords: machine learning, statistical modeling, match outcome prediction, association football, sports analytics, Elo rating system, Poisson distribution, Dixon-Coles model, multinomial logistic regression, MNLogit, XGBoost, Random Forests, Optuna, hyperparameter optimization, Log Loss, Brier Score, permutation feature importance, multicollinearity, sports econometrics

The aim of this study was to compare the performance of machine learning models and classical statistical estimators with baseline models in a three-class classification task of football match outcomes, where:

- 0 – home win
- 1 – draw
- 2 – away win

The study was conducted on a properly prepared dataset of football matches from 2000–2025 across various leagues, which initially contained approximately 230,000 observations.

Preprocessing such a large dataset required several fundamental changes. The `MatchTime` column was dropped because it contained over 50% missing data. Furthermore, a decision was made to exclude niche leagues containing significant amounts of missing data. Subsequently, post--match variables had to be removed to eliminate data leakage, along with the removal of records with missing labels. To improve predictive capability, derived features such as `EloDifference` were also introduced.

The next step involved training Poisson regression models with the Dixon-Coles adjustment, multiclass logistic regression, decision trees, random forests, and XGBoost. For hyperparameter optimization, the preprocessed dataset was split into training, validation, and test sets. This required modifying the approach by treating the data as a time series, since the outcomes of subsequent matches for a given team are interdependent and follow a specific chronological order. To find these optimal hyperparameter values, the `Optuna` framework [1] was utilized with the objective of minimizing the `LogLoss`. GPU acceleration was employed to speed up the training process.

The results of the implemented models, based on the selected feature set, demonstrated a slight improvement in the applied metrics compared to the baseline models. Furthermore, the performance of the machine learning models did not differ substantially from the statistical models, achieving similar results: `Accuracy` of approx. 50–51%, `LogLoss` of approx. 1.00, and a `Brier Score` of approx. 0.59–0.60. An analysis of the confusion matrices revealed that all models struggled to predict draws, which resulted in better metric values in leagues where statistically fewer matches end in a draw. The aforementioned derived feature, `EloDifference`, proved to be the most important, which indicates that with the current feature set, non-linear models offer only a slight improvement over linear models. In the case of classical statistical models, the significance of this variable was also reflected by a very low p-value and a high z-statistic. The obtained results confirm the high utility of the presented mathematical and statistical framework in the quantitative analysis and modeling of highly random sports processes.

Spis treści

Streszczenie **2**

Abstract **3**

Wykaz symboli i skrótów **6**

Spis rysunków **7**

Spis tabel **8**

1 Wstęp **9**
1.1 Motywacja 9
1.2 Cel i zakres pracy 9

2 Przegląd Literatury **11**
2.1 Znaczenie przypadku i jakości drużyny w modelowaniu wyników piłkarskich . . . 11
2.1.1 Początkowe błędne założenia 11
2.1.2 Wprowadzenie czynnika jakości drużyny 11
2.2 Ewolucja modeli opartych na rozkładzie Poissona 11
2.2.1 Niezależny model Mahera 11
2.2.2 Dwuwymiarowy rozkład Poissona 11
2.3 Dynamiczne podejście do formy zespołów i Korekta Niskich Wyników 12
2.3.1 Model Dixona i Colesa 12
2.3.2 Korekta remisów 12
2.3.3 Ważenie obserwacji w czasie i wygaszanie wag 13
2.4 Współczesne Modele Stochastyczne w Przestrzeni Stanów 13
2.4.1 Zmiana podejścia do formy zespołów 13
2.4.2 Główne założenia modelu Koopman-Lit 13
2.5 Efektywność na rynku zakładów jako metoda weryfikacji modelu 14
2.5.1 Marża bukmacherska i optymalna strategia gry 14
2.5.2 Odwrócony efekt faworyt-outsider i profesjonalizacja rynku bukmacherskiego 15

3 Podstawy Teoretyczne **16**
3.1 Modele klasyfikacyjne uczenia maszynowego 16
3.1.1 Regresja logistyczna wieloklasowa 16
3.1.2 Drzewa decyzyjne i lasy losowe 20
3.1.3 Gradient boosting i model XGBoost 23
3.2 Modele Statystyczne 27
3.2.1 Model Poissona 27
3.2.2 Regresja Logistyczna w ujęciu statystycznym 29
3.3 Metryki 30

4 Dane i metodyka **32**
4.1 Zbiór danych i jego charakterystyka . . . 32
4.2 Wstępna eksploracja danych . . . 32
4.3 Preprocessing i czyszczenie danych . . . 35
4.3.1 Duplikaty i niszowe zespoły . . . 35
4.3.2 Eliminacja wycieku danych . . . 35
4.3.3 Braki danych i selekcja lig . . . 35
4.3.4 Przygotowanie cech do modeli liniowych i regularyzacji . . . 36
4.4 Inżynieria cech . . . 37
4.4.1 Cechy oparte na rankingu Elo . . . 37
4.4.2 Cechy oparte na zmiennych Formy . . . 37
4.5 Architektura potoku . . . 38
4.5.1 Przetwarzanie i kodowanie zmiennych (preprocessing) . . . 38
4.5.2 Walidacja i podział danych . . . 38
4.5.3 Optymalizacja hiperparametrów . . . 39

5 Wyniki **41**
5.1 Modele odniesienia . . . 41
5.2 Porównanie skuteczności modeli uczenia maszynowego i statystycznych . . . 41
5.2.1 Modele uczenia maszynowego . . . 41
5.2.2 Modele statystyczne . . . 42
5.3 Zróżnicowanie skuteczności modeli . . . 43
5.3.1 Analiza macierzy pomyłek . . . 43
5.3.2 Porównanie jakości modeli w różnych ligach . . . 44
5.3.3 Ewolucja błędu `LogLoss` w czasie . . . 46
5.4 Analiza ważności cech . . . 46

Podsumowanie **48**

Bibliografia **50**

Spis załączników **52**

Załączniki **53**

Wykaz symboli i skrótów

Skróty literowe

NID	Niezależne o tożsamym rozkładzie normalnym (ang. *Normally and Independently Distributed*)
SSE	Suma kwadratów błędów (ang. *Sum of Squared Errors*)
CART	Algorytm drzew klasyfikacyjnych i regresyjnych (ang. *Classification and Regression Trees*)

Symbole matematyczne

α	Parametr proporcji między karą L_1 (Lasso) a L_2 (Ridge) w regularyzacji Elastic Net, gdzie wartość 1 oznacza karę L_1, a 0 karę L_2
α_i	Parametr określający siłę ataku drużyny i
β_j	Parametr określający słabość obrony drużyny j
$\hat{B}$	Estymator macierzy współczynników (parametrów) w regresji wieloklasowej
$\eta_{k,t}$	Losowe zakłócenie (szum) przejścia stanu dla parametru k w momencie t
$\hat{f}(x)$	Wektor funkcji predykcyjnych (prognoz) dla wektora cech x
$\hat{G}(x)$	Funkcja decyzyjna klasyfikatora (przypisująca obserwację do jednej z klas)
G_m	Indeks Giniego dla węzła (obszaru) m (miara nieczystości węzła)
γ	Parametr zależności między liczbami goli w modelu Koopmana-Lita
n	Liczba drużyn (zespołów) w analizowanych rozgrywkach
λ	Parametr regularyzacji (siły kary) w modelach uczenia maszynowego
$L(\beta)$	Funkcja wiarygodności (*likelihood function*) dla wektora parametrów β
$\ell(\beta)$	Logarytm funkcji wiarygodności (*log-likelihood function*) dla wektora parametrów β
r	Parametr progowy (próg rentowności) stosowany do selekcji zakładów o dodatniej wartości oczekiwanej
$p_k(x)$	Prawdopodobieństwo przynależności obserwacji o wektorze cech x do klasy k
R_m	m-ty obszar (węzeł) podziału przestrzeni cech w drzewie decyzyjnym
ρ	Parametr korelacji między liczbą bramek zdobytych przez gospodarzy i gości
$\sigma^2_{k,t}$	Wariancja losowego zakłócenia dla parametru k w momencie t
t	Moment czasowy, dla którego szacowana jest siła zespołów (dokonywana jest estymacja)
$\tau(x, y)$	Funkcja korygująca Dixona-Colesa dla niskich wyników bramkowych (0:0, 1:1, 1:0, 0:1)
z_k	Wartość kombinacji liniowej cech (aktywacja) dla klasy k przed zastosowaniem funkcji softmax

Spis rysunków

1 Rozkład liczby meczów w zależności od ligi 33
2 Porównanie rzeczywistego rozkładu liczby goli w meczu z teoretycznym rozkładem Poissona. 33
3 Analiza przewagi własnego boiska 34
4 Globalna ewolucja lig 34
5 Braki danych w surowym zbiorze 36
6 Rozkłady kilku przykładowych cech numerycznych w surowym zbiorze są bardzo zróżnicowane, dlatego konieczne jest ich przeskalowanie. 37
7 Macierze pomyłek modeli predykcyjnych 43
8 Porównanie błędu `LogLoss` w różnych ligach dla modeli uczenia maszynowego i statystycznych. 45
9 Ewolucja błędu `LogLoss` według roku w zbiorze testowym w latach 2023-2025 . . 46
10 Permutation Feature Importance dla modeli ML 47

Spis tabel

1 Konfiguracja potoków wykorzystanych w procesie preprocessingu dla modeli uczenia maszynowego 38
2 Wyniki optymalizacji hiperparametrów modeli uczenia maszynowego przy użyciu narzędzia Optuna [1] (na podstawie 50 prób dla każdego modelu) 40
3 Wyniki predykcji modeli referencyjnych na zbiorze testowym 41
4 Wyniki ewaluacji modeli uczenia maszynowego na zbiorze testowym 42
5 Wyniki ewaluacji modeli statystycznych oraz istotność statystyczna zmiennych . 42
6 Liczność zbioru testowego w podziale na poszczególne ligi piłkarskie 45

1 Wstęp

Piłka nożna jest jedną z najpopularniejszych dyscyplin sportowych na świecie. Najważniejsze wydarzenia w tej dyscyplinie, takie jak Mistrzostwa Świata czy Liga Mistrzów, przyciągają miliony fanów na stadiony i przed telewizory. W rezultacie jest ona wysoce skomercjalizowanym sektorem, w którym obraca się ogromnym kapitałem. Jedną z najważniejszych dziedzin tej komercjalizacji jest rynek bukmacherski, opierający się na przewidywaniu wyników meczów [1]. Skuteczność prognoz przekłada się w tym kontekście bezpośrednio na wymierne efekty ekonomiczne, co stanowi silny impuls do rozwoju metod ich generowania. W konsekwencji w ostatnich dekadach obserwuje się rosnące zainteresowanie zastosowaniem metod matematycznych, statystycznych oraz uczenia maszynowego do analizy i prognozowania wyników rywalizacji sportowej. Firmy bukmacherskie wiedzą, że każde, nawet najmniejsze, ulepszenie aktualnych modeli może w długofalowej perspektywie przynieść milionowy zysk. Konstrukcja konkurencyjnego modelu predykcji wyników meczów piłkarskich wiąże się ze specyficznymi trudnościami wynikającymi z dyskretnej natury zdobywanych goli oraz faktu, że jest to gra o relatywnie niskiej liczbie punktów.

1.1 Motywacja

Motywacją do podjęcia tego tematu była chęć połączenia wieloletniej pasji autora do piłki nożnej z praktycznym wykorzystaniem modeli uczenia maszynowego i metod statystycznych, które poznano w toku studiów licencjackich na kierunku matematyka i analiza danych. Co więcej, podjęty problem badawczy stanowi doskonałą ilustrację tego, jak zaawansowane narzędzia matematyczne mogą zostać z powodzeniem zaaplikowane do analizy zjawisk obserwowanych w życiu codziennym. Tego typu matematyka stosowana stanowi szczególny obszar zainteresowania, ponieważ ukazuje bezpośrednie przełożenie teorii akademickiej na praktyczne zagadnienia decyzyjne i analityczne. Ponadto, z perspektywy matematycznej, prognozowanie wyników meczów piłkarskich jest klasycznym problemem klasyfikacyjnym o trzech możliwych wynikach. Stanowi to doskonały punkt wyjścia do porównania skuteczności różnych klas modeli w warunkach danych rzeczywistych, które są obarczone naturalną niepewnością i wysokim poziomem szumu informacyjnego.

1.2 Cel i zakres pracy

Głównym celem pracy było zbudowanie i porównanie skuteczności predykcyjnej modeli statystycznych oraz nowoczesnych modeli uczenia maszynowego (m.in. XGBoost oraz lasów losowych) w zadaniu klasyfikacji trójklasowej wyniku meczu piłkarskiego (wygrana gospodarzy, remis, wygrana gości). Uzyskane wyniki odniesiono do modeli naiwnych, przyjętych jako punkt odniesienia np. heurystyki zakładającej, że mecz zawsze wygrywa gospodarz z wykorzystaniem szeregu metryk oceny jakości klasyfikacji.

Cele szczegółowe pracy obejmowały:

– zestawienie klasycznych modeli statystycznych opartych na rozkładzie Poissona z podejściami wykorzystującymi uczenie maszynowe w spójnych warunkach eksperymentalnych,

[1] Rozwinięciem tej podstawowej oferty są zakłady na liczbę strzelonych goli, liczbę kartek czy statystyki indywidualne zawodników, wprowadzone przez bukmacherów w kolejnych latach w celu poszerzenia oferty.

– zbadanie, w jakim stopniu jakość predykcji poszczególnych modeli zmienia się wraz z upływem kolejnych sezonów,

– porównanie skuteczności modeli pomiędzy różnymi ligami piłkarskimi,

– ocenę, w jakim stopniu ogólna skuteczność predykcji jest zaniżana przez wyniki uzyskiwane w ligach niszowych.

Zakres pracy obejmuje analizę danych historycznych z około 130 tysięcy meczów rozegranych w kilku europejskich ligach piłkarskich, w tym Ekstraklasie, Premier League oraz La Liga. Do budowy modeli wykorzystano statystyki historyczne drużyn (m.in. ratingi Elo). Kursy bukmacherskie, pomimo obecności w surowym zbiorze danych, nie zostały wykorzystane w modelach. Proces przetwarzania danych i trenowania zrealizowano w środowisku obliczeniowym z użyciem GPU.

2 Przegląd Literatury

2.1 Znaczenie przypadku i jakości drużyny w modelowaniu wyników piłkarskich

2.1.1 Początkowe błędne założenia

Wczesne badania koncentrowały się na oszacowaniu wpływu szumu losowego na wynik meczów piłkarskich. W pierwszych pracach Moroney (1951, cyt. za: Maher [13]) oraz Reep, Pollard i Benjamin (1971, cyt. za: Maher [13]) błędnie uważali, że wynik meczu jest kwestią przypadku. Ich zdaniem najlepiej opisywał go rozkład ujemny dwumianowy, w którym nie miało znaczenia, jakie drużyny grają przeciwko sobie. [13]

2.1.2 Wprowadzenie czynnika jakości drużyny

W późniejszym okresie powyższe podejście zostało zakwestionowane przez Hilla (1974, cyt. za: Maher [13]). Pokazał on, że można przewidzieć, z pewną dokładnością, końcowy układ tabeli. Dowodziło to, że o ile wynik pojedynczego meczu może być dziełem przypadku, o tyle w skali całego sezonu widać wyraźną korelację między jakością danej drużyny a osiąganymi wynikami. Pionierzy badań matematycznych dotyczących piłki nożnej popełnili błąd statystyczny. Choć wyniki pojedynczej drużyny mają rozkład Poissona, to różne drużyny mają różne intensywności. Ignorując różnice między zespołami, badacze otrzymali mieszany rozkład Poissona, który przypomina rozkład ujemny dwumianowy. Przed rewolucyjną pracą Mahera (1982) pojawiły się jeszcze badania Thompsona (1975, cyt. za: Maher [13]), który jako pierwszy zastosował rachunek prawdopodobieństwa do oceny siły drużyny, budując w tym celu funkcję największej wiarygodności, a także praca Harville'a (1977, cyt. za: Maher [13]), który zaproponował zastosowanie do tego problemu modeli liniowych. [5] [13]

2.2 Ewolucja modeli opartych na rozkładzie Poissona

2.2.1 Niezależny model Mahera

Przełomem w modelowaniu wyników poszczególnych meczów piłkarskich była praca Mahera (1982). Założył on, że liczbę bramek zdobytych przez gości i gospodarzy można modelować jako niezależne zmienne o rozkładzie Poissona. Wprowadził też oznaczenia α_i i β_j, odpowiednio jako siłę ataku danej drużyny i słabość obrony przeciwników, a parametry te estymował metodą największej wiarygodności. Już na tym wczesnym etapie badań wprowadził również parametr k jako współczynnik proporcjonalności globalnej przewagi własnego boiska. Służył on do powiązania parametrów wyjazdowych z parametrami domowymi: obniżał siłę ataku na wyjeździe oraz zmniejszał słabość obrony w meczach domowych. W późniejszych pracach parametr ten był nazywany współczynnikiem przewagi własnego boiska. [13]

2.2.2 Dwuwymiarowy rozkład Poissona

W piątym rozdziale swojej pracy Maher doszedł do wniosku, z którego korzystało później wielu badaczy zajmujących się modelowaniem matematycznym wyników meczów piłkarskich. Zauważył, że konieczne jest użycie dwuwymiarowego rozkładu Poissona, argumentując to dwoma powodami. Pierwszy powód miał charakter taktyczny: obie drużyny nie grają na oddzielnych

boiskach, lecz na tym samym, przez co ewentualne osłabienia czy zmiana strategii na bardziej ofensywną przez drużynę przegrywającą wpływają na prawdopodobieństwo zdobycia bramki dla obu zespołów jednocześnie. Drugi powód miał charakter statystyczny: na przestrzeni trzech analizowanych sezonów w czterech dywizjach lig angielskich model oparty na klasycznej niezależności systematycznie niedoszacowywał liczby remisów, co sugerowało korelację między liczbą bramek zdobywanych przez gospodarzy i gości. Dopasowanie do danych parametru korelacji $\rho = 0,2$ pomogło rozwiązać ten problem i poprawiło jakość modelu. [13]

2.3 Dynamiczne podejście do formy zespołów i Korekta Niskich Wyników

2.3.1 Model Dixona i Colesa

Już na samym początku swojej pracy autorzy zaznaczyli, że ich zdaniem model Mahera wymaga poprawy, gdyż nie uwzględnia aktualnej formy drużyn – co z logicznego punktu widzenia wydaje się istotnym ograniczeniem, ponieważ np. zmiana trenera czy odejście kluczowego sponsora może wpłynąć na wyniki osiągane przez drużynę. Badacze wykorzystali dane z sezonów 1992/93, 1993/94 i 1994/95 i zaczęli od policzenia częstotliwości strzelenia danej liczby bramek u siebie i na wyjeździe, a następnie sprawdzili, czy faktycznie rozkład Poissona dobrze je opisuje. Po potwierdzeniu tej hipotezy przystąpili do weryfikacji założenia o niezależności zmiennych.

Porównując zebrane dane, zauważyli, że istnieje przybliżona niezależność, poza czterema niskimi rezultatami (0:0, 1:1, 1:0, 0:1): bezbramkowe remisy 0:0 pojawiały się w badanej próbie o 11,5% częściej, remisy 1:1 o 5,7% częściej niż zakładał model, podczas gdy rzeczywista liczba rozstrzygnięć 1:0 oraz 0:1 była odpowiednio o 6,3% i 8,0% niższa od teoretycznych szacunków. [2] Dlatego uznali, że zamiast dwuwymiarowego rozkładu Poissona, wystarczy jedynie użyć poprawki $\tau(x, y)$, która sztucznie zwiększy prawdopodobieństwo dla 0:0 i 1:1, a obniży dla 1:0 i 0:1 [5].

2.3.2 Korekta remisów

Punktem wyjścia dla poruszonej w powyższej sekcji korekty remisów było wprowadzenie przez Dixona i Colesa zmodyfikowanej gęstości łącznej dla dwóch niezależnych zmiennych o rozkładzie Poissona, opartej na dodatkowym parametrze zależności ρ [3]. Domyślnie jego wartość jest ustawiona na 0, co odpowiada niezależności obu zmiennych.

Następnie autorzy zdecydowali się wprowadzić ograniczenie na wielkość parametrów w modelu, aby ułatwić modelowi znalezienie optymalnego rozwiązania przy maksymalizacji funkcji wiarygodności[4]:

$$\frac{1}{n}\sum_{i=1}^{n}\alpha_i = 1 \tag{1}$$

[2] Warto zaznaczyć, że częstość występowania wyższych wyników, np. 6:2, była niedoszacowana w jeszcze większym stopniu (tutaj o 46,4%). Autorzy wyznaczyli jednak również błąd standardowy, który dla wspomnianych czterech niskich rozstrzygnięć oscylował w okolicy 2-3, podczas gdy dla wyników rzadszych był on znacznie większy (np. 41,33 dla wyniku 6:2).

[3] Pełną postać funkcji korygującej oraz definicje parametrów przedstawiono w podrozdziale Podstawy Teoretyczne.

[4] W zasadzie ograniczamy tylko wielkość parametru ataku, jednak, jako że oczekiwana liczba zdobytych goli przez gości i gospodarzy stanowi iloczyn obu opisanych parametrów oraz, dla gospodarzy, parametru przewagi własnego boiska, wymusza to ograniczenie na drugi parametr.

Autorzy zauważyli, że wprowadzenie parametru zależności ρ uniemożliwia wyznaczenie analitycznej postaci rozwiązania problemu maksymalizacji funkcji wiarygodności i zmusza do szukania rozwiązań metodami numerycznymi [5].

2.3.3 Ważenie obserwacji w czasie i wygaszanie wag

Badacze wskazali, że utrzymanie stałej wartości parametrów α i β jest nierealne, a największy wpływ na aktualną formę drużyny mają ostatnie mecze. Autorzy zastosowali mechanizm wygaszania w czasie, który przypisywał większe znaczenie wynikom niedawnych spotkań, stopniowo marginalizując wpływ starszych meczów na aktualne szacunki siły zespołów. Uzyskali to za pomocą modyfikacji funkcji wiarygodności na funkcję "pseudowiarygodności" dla każdego okresu czasowego t, przy czym maksymalizacja dla danego momentu t bazowała wyłącznie na meczach z przeszłości. Aby mechanizm starzenia się informacji był skuteczny, Dixon i Coles odrzucili koncepcję binarnej funkcji odcięcia starych meczów na rzecz płynnej funkcji wykładniczej. Co niezwykle istotne z perspektywy współczesnej analityki danych, autorzy uzależnili optymalizację tempa zapominania od zdolności predykcyjnych modelu względem ostatecznego rozstrzygnięcia meczu (zwycięstwo gospodarzy, remis, zwycięstwo gości), a nie samych rozkładów bramek. Do wyznaczenia optymalnego parametru użyli funkcji opartej na logarytmie prawdopodobieństw przypisanych rzeczywistym wynikom. Stanowiło to wczesną formę zastosowania metryki `LogLoss`, która do dziś pozostaje jednym z głównych standardów ewaluacji modeli probabilistycznych w klasyfikacji wieloklasowej. [5]

2.4 Współczesne Modele Stochastyczne w Przestrzeni Stanów

2.4.1 Zmiana podejścia do formy zespołów

W 2015 roku Koopman i Lit zaproponowali nowe podejście do przewidywania wyników meczów piłkarskich. W tym celu przeanalizowali spotkania z 9 sezonów angielskiej Premier League (od 2003/2004 do 2011/2012). Uznali oni sztuczną metodę wygaszania zaproponowaną przez Dixona i Colesa za zbyt sztywną i nienaturalną. Stwierdzili, że forma sportowa zespołu nie zmienia się według z góry narzuconej funkcji czasu, lecz ma charakter losowy. Drużyna może nagle stracić formę z powodu kontuzji, zmiany trenera czy kryzysu w szatni, a potem niespodziewanie ją odzyskać. Aby właściwie opisać to zjawisko, potrzebny był proces losowy [12].

2.4.2 Główne założenia modelu Koopman-Lit

Opracowany przez nich model był oparty na kilku głównych rozszerzeniach dotychczasowych pomysłów. Po pierwsze, autorzy odrzucili założenie Dixona & Colesa o niezależności bramek zdobywanych przez obie drużyny. Badacze zastosowali dwuwymiarowy rozkład Poissona z parametrem zależności γ. To bardzo praktyczne założenie: odzwierciedla ono fakt, że styl oraz intensywność gry jednej drużyny w trakcie meczu bezpośrednio wpływają na postawę rywala (np. gdy faworyt niespodziewanie przegrywa, zaczyna grać bardziej ryzykownie, co zwiększa szanse na kolejne bramki dla obu stron). Następnie zaproponowali podejście do formy jako zmiennej w przestrzeni stanów. W ich koncepcji parametry siły ofensywnej $\alpha_{i,t}$ danej drużyny oraz siły defensywnej $\beta_{j,t}$ przeciwnika przestały być stałe i zaczęto je traktować jako procesy

stochastyczne. Za zmiany stanów po danym tygodniu ligowym odpowiadał parametr

$$\eta_{k,t} \sim \text{NID}(0, \sigma^2_{k,t}), \quad \text{dla } k = \alpha, \beta, t = 1, \ldots, T \tag{2}$$

gdzie NID oznacza, że $\eta_{k,t}$ są niezależnymi zmiennymi losowymi o rozkładzie normalnym.

Kolejną istotną innowacją było uwzględnienie letniego i zimowego okna transferowego. To właśnie w tym czasie forma drużyny zmienia się najmocniej, ponieważ jest to czas przerwy od meczów ligowych i właśnie wtedy dochodzi do roszad kadrowych zespołów.[5] Aby zminimalizować błąd predykcji spowodowany tą przerwą, Koopman i Lit dopuścili zwiększenie wariancji dla zmiennej $\eta_{k,t}$ podczas pierwszych meczów nowego sezonu, natomiast przerwa zimowa okazała się nie mieć statystycznie istotnego wpływu. Następnym ciekawym rozszerzeniem był pomysł wprowadzenia oddzielnego parametru przewagi własnego boiska dla różnych grup drużyn. Autorzy uznali, że czołowe kluby z Premier League, które osiągają najlepsze wyniki, zyskują najwięcej z gry na własnym boisku. Najczęściej to właśnie takie drużyny mają największe stadiony, które co mecz wyprzedają prawie wszystkie bilety. Dla piłkarzy drużyny gości gra na takim obiekcie to zawsze więcej stresu, co może skutkować większą liczbą błędów. Jednak po przeprowadzeniu testów statystycznych okazało się, że nie jest to hipoteza istotna statystycznie i konieczne było jej odrzucenie. Ostatnim istotnym udoskonaleniem było zastosowanie tzw. *importance sampling*, ponieważ niegaussowski charakter modelu (wynikający z poissonowskiego rozkładu liczby goli) wymaga bardziej zaawansowanych metod estymacji. Polega on na zastąpieniu losowania ścieżki z rozkładu teoretycznego losowaniem ze specjalnie przygotowanego rozkładu pomocniczego, który jest budowany w oparciu o rzeczywiste, zaobserwowane wyniki meczów. [12]

2.5 Efektywność na rynku zakładów jako metoda weryfikacji modelu

Weryfikacja skuteczności modeli predykcyjnych w analityce sportowej rzadko opiera się wyłącznie na standardowych metrykach statystycznych, takich jak `Accuracy` czy `Logloss`. W praktyce badawczej naturalnym i najbardziej rygorystycznym sprawdzianem dla tworzonych algorytmów jest rynek zakładów bukmacherskich. Kursy oferowane przez operatorów zakładów nie są dziełem przypadku, lecz także opierają się na modelach matematycznych i prawdopodobieństwie. Zbudowanie modelu o skuteczności predykcyjnej przewyższającej systemy stosowane przez firmy bukmacherskie otwierałoby możliwość generowania systematycznych zysków finansowych, dzięki uzyskaniu dodatniej wartości oczekiwanej stawianych zakładów.

2.5.1 Marża bukmacherska i optymalna strategia gry

W zakładach bukmacherskich widoczne kursy nie są faktycznym odzwierciedleniem prawdopodobieństwa. Bukmacherzy doliczają marżę, przez co potencjalny gracz znajduje się w niekorzystnej sytuacji wyjściowej. Dixon i Pope na podstawie danych z 1993-1996 z 4 profesjonalnych lig z Wielkiej Brytanii wyliczyli w swojej pracy, że w klasycznych zakładach typu 1X2 marża u brytyjskich bukmacherów wynosiła średnio około 11%, natomiast w zakładach na dokładny wynik marża ta wynosi średnio aż 26%. Następnie znaleźli pomysł na obejście 9% podatku od zakładów bukmacherskich, jaki obowiązywał wtedy w Wielkiej Brytanii. Uznali, że zamiast obstawiać trzy oddzielne mecze, z których każdy ma oczekiwaną wypłatę przed opodatkowaniem równą 1,09,

[5] W modelu analizowane są wyłącznie rozgrywki ligowe. Choć nowoczesne rynki bukmacherskie obejmują również spotkania towarzyskie, ich trafne prognozowanie jest znacznie trudniejsze, głównie ze względu na mniejsze zaangażowanie zespołów oraz dużą liczbę rotacji kadrowych i nadrzędnych założeń taktycznych.

korzystniejsze jest połączenie ich w jeden kupon akumulowany, który daje oczekiwany zwrot przed podatkiem rzędu $1,09^3 = 1,30$. Jednak taka gra cechuje się wyższą wariancją, a więc jest bardziej ryzykowna. Autorzy zdecydowali się wprowadzić próg rentowności dla danego zakładu:

$$r = P_{\text{model}} \times \text{kurs bukmachera} \tag{3}$$

i grać tylko powyżej ustalonego progu. Zauważyli, że zysk zaczynał się na poziomie 1,2, przy czym był on największy dla $r = 1,4$ i wtedy osiągali średni zwrot na poziomie 20% z każdego postawionego funta. Jednak dla tego progu zakładów jest bardzo mało, ponieważ tak duże błędy bukmacherów zdarzają się rzadko. Co więcej, autorzy zauważają, że przy wysokich progach błąd standardowy szacunków i przedziały ufności (wyznaczone metodą bootstrapu) mocno się rozszerzają, a co za tym idzie, pojedyncza seria przegranych może spowodować większe wahania kapitału w krótkim okresie niż dla mniejszej wartości progu.[6]

2.5.2 Odwrócony efekt faworyt-outsider i profesjonalizacja rynku bukmacherskiego

Dla większości zdarzeń bukmacherskich gracze chętnie obstawiają mało prawdopodobne, pojedyncze zdarzenia o wysokich kursach. Liczą, że w ten sposób szybko się wzbogacą. Bukmacher, aby zabezpieczyć się przed dużą stratą, celowo obniża kursy takich wyników. Dixon i Pope odkryli, że na angielskim rynku piłkarskim jest odwrotnie: prawdopodobieństwa wygranej outsiderów są niedoszacowywane, a dla faworytów przeszacowywane. Autorzy przeanalizowali także inną metodę zarobku na rynku zakładów, tzw. arbitraż bukmacherski. Sposób ten polegał na porównywaniu kursów oferowanych przez różnych operatorów zakładów i dążeniu do sytuacji, w której przy wyborze najkorzystniejszych opcji dla każdego ze zdarzeń 1X2 suma prawdopodobieństw im odpowiadających będzie mniejsza niż 1:

$$\sum \frac{1}{\text{Kurs}} < 1 \tag{4}$$

W takiej sytuacji niezależnie od wyniku gracz zarabia. Badacze porównali rynek lat 80. z rynkiem lat 90. i zauważyli znaczącą transformację. Podczas gdy we wcześniejszym okresie udało im się znaleźć wiele takich sytuacji, tak dla danych analizowanych w opisywanej pracy nie było ani jednego takiego zdarzenia. Za główne powody podali ulepszenie modeli bukmacherskich oraz potencjalną nieformalną umowę między firmami, aby nie oddawać zysku graczom. [6]

3 Podstawy Teoretyczne

Poniższy rozdział służy porównaniu wykorzystanych w kodzie algorytmów klasyfikacji trójklasowej (wygrana gospodarzy, remis, wygrana gości), w której zmienna objaśniana ma charakter jakościowy. W badaniach zdecydowano się porównać trzy różne podejścia do modelowania: zaawansowane algorytmy uczenia maszynowego, klasyczne modele statystyczne oraz modele naiwne. Choć podejścia te wywodzą się z różnych epok i środowisk akademickich, u ich podstaw leżą spójne koncepcje matematyczne dotyczące statystyki i rachunku prawdopodobieństwa. Wybór tak zróżnicowanych metod pozwala na szczegółowe zbadanie kompromisu pomiędzy elastycznością modelu a interpretowalnością i wariancją. Klasyczne modele statystyczne nakładają silne założenia strukturalne na postać funkcji decyzyjnej, co skutkuje mniejszą elastycznością, lecz zapewnia wysoką stabilność (niską wariancję) oraz łatwość interpretacji wpływu poszczególnych zmiennych objaśniających. Z kolei złożone, nieparametryczne algorytmy uczenia maszynowego charakteryzują się wysoką elastycznością, co pozwala im na precyzyjne dopasowanie się do danych, jednak niesie ze sobą ryzyko przeuczenia (ang. *overfitting*) oraz ogranicza bezpośrednią interpretację faktycznych zależności w danych. Wprowadzenie do analizy modeli naiwnych oraz modeli punktu odniesienia (ang. *baseline models*) pozwala z kolei na ocenę rzeczywistego zysku predykcyjnego, jaki można otrzymać przez zastosowanie bardziej skomplikowanych struktur obliczeniowych. Kluczowym elementem procedury badawczej było rzetelne oszacowanie zdolności predykcyjnej modeli, czyli ich skuteczności na nowych, wcześniej niewidzianych obserwacjach. Jak powszechnie wskazuje się w literaturze, błąd obliczony na zbiorze treningowym jest miarą nadmiernie optymistyczną i nie może służyć jako wiarygodny wskaźnik jakości predykcji. W związku z tym jakość modeli została zweryfikowana na podstawie chronologicznego podziału danych na zbiór treningowy, walidacyjny i testowy. Do wszechstronnej oceny jakości klasyfikacji trójklasowej wykorzystano zestaw komplementarnych metryk oceny predykcji: dokładność (ang. accuracy), `LogLoss`, `Brier Score` oraz macierze pomyłek. [9, 11]

3.1 Modele klasyfikacyjne uczenia maszynowego

Wykorzystane w pracy metody uczenia maszynowego należą do klasy metod uczenia nadzorowanego. Podejście to opiera się na trenowaniu modelu na danych, gdzie każdy rekord z przestrzeni cech X posiada prawdziwą etykietę w zbiorze klas Y. W rozważanym problemie cechy macierzy X stanowią przedmeczowe statystyki dotyczące drużyn rozgrywających ze sobą mecz. Natomiast wektor Y to zakodowany wynik meczu. Celem projektu jest stworzenie modelu, który jak najlepiej odwzorowuje ukryte zależności, w tym te nieliniowe.

3.1.1 Regresja logistyczna wieloklasowa

Zanim przejdziemy do opisu Regresji Logistycznej w zadaniu klasyfikacji trójklasowej, zacznijmy od wyjaśnienia dlaczego zwykła regresja liniowa nie nadaje się do tego zadania. Rozpatrzmy równanie regresji wielokrotnej:

$$\hat{Y} = X_0\beta_0 + X_1\beta_1 + X_2\beta_2 + \cdots + X_p\beta_p = X\beta \tag{5}$$

gdzie:

– $\hat{Y}$ – wektor wartości przewidywanych dla wszystkich N obserwacji,

– X – macierz zmiennych objaśniających,

– β – wektor szukanych parametrów (współczynników modelu).

Załóżmy przy tym, że Y to trójklasowa zmienna kategorialna:

$$Y = \begin{cases} 0 & \text{jeśli wygrywa gospodarz,} \\ 1 & \text{jeśli remis,} \\ 2 & \text{jeśli wygrywają goście.} \end{cases} \tag{6}$$

Dla modelu liniowego klasyczną miarą dopasowania jest Metoda Najmniejszych Kwadratów:

$$\mathrm{SSE}(\beta) = \sum_{i=1}^{N} \left(y_i - x_i^T \beta\right)^2 \tag{7}$$

Natomiast zapis macierzowy rozwiązania zadania minimalizacji SSE ma postać:

$$\hat{\beta} = (X^T X)^{-1} X^T Y \tag{8}$$

Jednak napotyka się tutaj podstawowy problem: metoda MNK minimalizuje sumę kwadratów odległości prawdziwych wartości y_i od prognoz $x_i^T \beta$. Gdy podstawiamy za Y $\{0, 1, 2\}$, algorytm nie interpretuje wprowadzonych wartości jako zakodowanych nazw wyników i traktuje je jak zwykłe liczby rzeczywiste. Model MNK zakłada, że różnica między remisem (1) a wygraną gospodarzy (0) jest identyczna jak między wygraną gości (2) a remisem (1). Co więcej, takie kodowanie sztucznie narzuca porządek oraz jednakowe odległości między kategoriami. Wobec tego zmiana oznaczeń klas (np. przypisanie remisowi wartości 2) dałaby całkowicie inne, sprzeczne wyniki.[11, s. 137–138]

Aby naprawić ten problem naturalnym rozwiązaniem wydaje się być one-hot encoding klas polegający na stworzeniu macierzy odpowiedzi $Y_{N\times 3}$. Wówczas wektor prognoz przyjmuje wartość

$$\hat{f}(x)^T = x^T \hat{B} \tag{9}$$

W regresji na macierzy wskaźników dla trzech klas ($K = 3$) dopasowuje się trzy niezależne równania liniowe:

$$\begin{aligned} \hat{f}_0(x) &= x^T \beta_1 \quad \text{(zwycięstwo gospodarzy)} \\ \hat{f}_1(x) &= x^T \beta_2 \quad \text{(remis – klasa środkowa)} \\ \hat{f}_2(x) &= x^T \beta_3 \quad \text{(zwycięstwo gości)} \end{aligned} \tag{10}$$

Decyzję o klasyfikacji podejmuje się za pomocą reguły *argmax*:

$$\hat{G}(x) = \arg \max_{k \in \{0,1,2\}} \hat{f}_k(x) \tag{11}$$

Jednak napotyka się tutaj następujący problem: Regresja liniowa dąży do minimalizacji funkcji celu za pomocą prostych linii. Jako, że środkowa klasa (remisy) ma 1 na środku wykresu i 0 na obu końcach (dla wykresu względem zakodowanej zmiennej odpowiadającej za wynik meczu), to linia regresji byłaby ciągnięta do wartości 0 po obu stronach. Przez co linia regresji byłaby mocno spłaszczona i leżałaby poniżej obu klas skrajnych. Dlatego model nie przewidywałby

nigdy remisów. [9, s. 104-106]

Chcemy więc zmodyfikować podejście tak, aby wartości z przedziału $(-\infty, \infty)$ można było interpretować jako prawdopodobieństwa przynależności do każdej z trzech klas. Użyjemy do tego regresji logistycznej. Zacznijmy od klasycznej wersji z odpowiedzią binarną. Zdefiniujmy:

$$p(x) = \frac{e^{x^T\beta}}{1 + e^{x^T\beta}} = P(Y = 1 \mid X = x) \tag{12}$$

Oczywiście analogicznie zdefiniowane jest prawdopodobieństwo przynależności do klasy 0:

$$P(Y = 0 \mid X = x) = 1 - p(x) = \frac{1}{1 + e^{x^T\beta}} \tag{13}$$

Kolejną ważną definicją jest szansa, która jest wstępem do koniecznego matematycznie przejścia do postaci liniowej:

$$\text{Szansa} = \frac{p(X)}{1 - p(X)} = e^{x^T\beta} \tag{14}$$

Szansa w modelu regresji logistycznej oznacza stosunek prawdopodobieństwa $Y = 1$ względem $Y = 0$ pod warunkiem naszych danych. Aby wrócić do postaci liniowej zdefiniowany został logarytm szansy:

$$\text{Log-odds} = \ln\left(\frac{p(X)}{1 - p(X)}\right) = x^T\beta \tag{15}$$

Pozwala on łatwiej zrozumieć interpretację poszczególnych zmiennych. Wzrost wartości zmiennej x_j o jedną jednostkę przy założeniu, że wartości wszystkich pozostałych cech nie ulegną zmianie powoduje zmianę logarytmu szans o wartość β_j. [11, s. 139-140]

Metodą estymacji parametrów w modelu logistycznym, w odróżnieniu od modelu liniowego, jest metoda największej wiarygodności [11, s. 140-141]. Wywodzi się od iloczynu prawdopodobieństw dla pojedynczych obserwacji:

$$L(\beta) = \prod_{i=1}^{N} p(x_i)^{y_i}(1 - p(x_i))^{1-y_i} \tag{16}$$

Po przyłożeniu logarytmu, podstawieniu wzoru na sigmoidę i przekształceniach, log-wiarygodność dla wektora parametrów przybiera postać:

$$\ell(\beta) = \sum_{i=1}^{N} \left\{ y_i x_i^T\beta - \log(1 + e^{x_i^T\beta}) \right\} \tag{17}$$

W opisanym problemie konieczne było rozszerzenie tego do trójklasowej zmiennej odpowiedzi. W implementacjach modeli uczenia maszynowego używamy do tego funkcji Softmax. Zapis ten jest matematycznie równoważny klasycznemu podejściu statystycznemu. Dla każdej klasy $k \in \{0, 1, 2\}$ definiowana jest najpierw wartość funkcji liniowej:

$$z_k = x^T\beta_k \tag{18}$$

gdzie β_k to wektor parametrów przypisany do klasy k, a x to wektor zmiennych objaśniających.

Prawdopodobieństwo, że dana obserwacja x należy do klasy k, wyraża się za pomocą nieli-

niowej transformacji tych wartości, wykorzystującej funkcję Softmax:

$$p_k(x) = P(Y = k \mid X = x) = \frac{e^{z_k}}{\sum_{l=0}^{2} e^{z_l}} = \frac{e^{x^T \beta_k}}{\sum_{l=0}^{2} e^{x^T \beta_l}} \tag{19}$$

Dzięki dodatniości funkcji eksponentalnej oraz normalizacji mamy gwarancję, że $p_k(x) \in [0, 1]$. Ponadto funkcja Softmax jest niezmiennicza względem dodawania skalarów. [11, s. 145–146]

Współczynniki funkcji Softmax dobierane są poprzez minimalizację ujemnej funkcji log-wiarygodności tzw. Entropii Krzyżowej [11, s. 405]:

$$\mathcal{L}(\beta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=0}^{2} y_{ik} \log(p_k(x_i)) \tag{20}$$

gdzie:

- N – całkowita liczba obserwacji w zbiorze danych,
- y_{ik} – zmienna wskaźnikowa przyjmująca wartość 1, jeśli i-ty mecz zakończył się wynikiem klasy k, oraz 0 w przeciwnym razie,
- $p_k(x_i)$ – wyliczone przez model prawdopodobieństwo, że i-ty mecz zakończy się wynikiem klasy k.

Jednak modelowanie meczów piłkarskich wiąże się z pewnymi problemami, których sama funkcja Softmax nie jest w stanie rozwiązać:

- Część cech wykazuje przybliżoną współliniowość (tutaj np. forma z 3 i 5 ostatnich meczów).
- Zbiór danych zawiera relatywnie dużą liczbę cech, co bez odpowiedniej kontroli sprzyja przeuczeniu modelu.

Aby zapobiec przeuczeniu, wprowadza się regularyzację typu Elastic Net [9, s. 662–663]. Warto wskazać, że o ile w cytowanym źródle metoda ta jest omawiana głównie w kontekście problemów wysokowymiarowych (gdzie liczba cech znacznie przewyższa liczbę obserwacji), o tyle w niniejszej pracy mamy do czynienia z klasycznym przypadkiem niskowymiarowym ($p \ll N$). Ponadto, aby uniknąć kolizji oznaczeń z parametrem przewagi własnego boiska k (wprowadzonym wcześniej w modelach Poissona), w poniższym sformułowaniu indeks c odnosi się bezpośrednio do trzech klasyfikowanych klas wyników meczu ($c \in \{0, 1, 2\}$):

$$\mathcal{L}_{\text{reg}}(\beta) = \mathcal{L}(\beta) + \lambda \sum_{c=0}^{2} \sum_{j=1}^{p} \left(\alpha |\beta_{cj}| + (1 - \alpha) \beta_{cj}^2 \right) \tag{21}$$

gdzie:

- $\mathcal{L}(\beta)$ – wyjściowa strata entropii krzyżowej,
- $\lambda \geq 0$ – parametr kontrolujący ogólną siłę regularyzacji,
- α – współczynnik proporcji określający udział kary L_1 (Lasso), odpowiadający parametrowi *l1_ratio* w implementacji biblioteki *scikit-learn* [11, s. 273–274],

– β_{cj} – waga (parametr) dla j-tej cechy i c-tej klasy.

Dla $\alpha = 1$ otrzymujemy czystą karę L_1 (Lasso), która ma zdolność zerowania nieistotnych współczynników, natomiast dla $\alpha = 0$ otrzymujemy czystą karę L_2 (Ridge). Zaprezentowane sformułowanie wieloklasowe (multinomialne) regularyzacji opiera się na klasycznym ujęciu podręcznikowym [9, s. 662–663], przy czym parametr α określa wagę kary L_1, co jest bezpośrednio spójne z implementacją w pakietach obliczeniowych języka Python, takich jak *scikit-learn* [11, s. 273–274].

3.1.2 Drzewa decyzyjne i lasy losowe

Kolejnymi metodami użytymi w kodzie były drzewo decyzyjne i las losowy.

Pojedyncze drzewo zbudowane jest poprzez podziały przestrzeni, które w danym momencie generują maksymalny zysk. W każdym węźle drzewa definiuje się proporcję wystąpienia klasy $k \in \{0, 1, 2\}$ za pomocą wzoru[9, s. 307]:

$$\hat{p}_{mk} = \frac{1}{N_m} \sum_{i:x_i \in R_m} I(y_i = k) \tag{22}$$

gdzie:

– $\hat{p}_{mk}$ – proporcja obserwacji należących do klasy k w obszarze (węźle) m, stanowiąca estymator prawdopodobieństwa,

– N_m – całkowita liczba obserwacji, które wpadły do obszaru R_m, [11, s. 338]

Następnie algorytm przeszukuje zachłannie wszystkie możliwe na danym kroku podziały, tak aby otrzymany zysk był możliwie największy, tutaj jest nim możliwie największa redukcja poziomu nieczystości węzła. W kodzie narzędzie Optuna [1] [6] wybiera optymalny z możliwych kryteriów podziału w celu minimalizacji `LogLoss` na zbiorze walidacyjnym:

Pierwszym z nich jest Indeks Giniego [9, s. 309]:

$$G_m = \sum_{k=0}^{2} \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=0}^{2} \hat{p}_{mk}^2 \tag{23}$$

Drugą miarą jest entropia [9, s. 309]:

$$D_m = -\sum_{k=0}^{2} \hat{p}_{mk} \log \hat{p}_{mk} \tag{24}$$

gdzie:

– G_m – wskaźnik zanieczyszczenia Giniego dla węzła m,

– D_m – entropia dla węzła m,

– k – indeks klasy, przyjmujący wartości ze zbioru $\{0, 1, 2\}$,

– $\hat{p}_{mk}$ – proporcja (wyestymowane prawdopodobieństwo) obserwacji należących do klasy k w węźle m.

[6] Więcej o tym frameworku w sekcji Dane i Metodyka

Entropia pochodzi z teorii informacji i mierzy poziom chaosu w węźle. Docelowo poszukiwany podział ma mieć jak najmniejszą wartość indeksu Giniego bądź entropii. [11, s. 338–340] Wskazane miary stanowią podstawę do wyznaczenia optymalnych granic podziału przestrzeni cech. Proces ten realizowany jest za pomocą rekurencyjnego podziału binarnego. [11, s. 334–335] W bibliotece scikit-learn zastosowano do tego algorytm CART, który w każdym węźle poszukuje zmiennej objaśniającej x_j oraz punktu podziału s, dzielących węzeł m na dwa obszary:

$$L(j, s) = \{x \mid x_j \leq s\} \quad \text{oraz} \quad R(j, s) = \{x \mid x_j > s\} \tag{25}$$

gdzie:

- x_j – wybrana zmienna objaśniająca z wektora obserwacji x, na podstawie której następuje podział,
- s – punkt podziału (wartość rozgraniczająca) dla wybranej cechy x_j,

Wybór pary (j, s) następuje poprzez minimalizację sumy [9, s. 309]:

$$\arg\min_{j,s} \left[\frac{N_{mL}}{N_m} Q_L(j, s) + \frac{N_{mR}}{N_m} Q_R(j, s) \right] \tag{26}$$

gdzie:

- N_m – całkowita liczba obserwacji w aktualnie dzielonym węźle m,
- N_{mL}, N_{mR} – liczba meczów trafiających odpowiednio do lewego (L) i prawego (R) węzła powstałego wskutek podziału ($N_m = N_{mL} + N_{mR}$),
- $Q_L(j, s), Q_R(j, s)$ – miary wyliczone odpowiednio dla lewego i prawego węzła potomnego przy podziale względem zmiennej j w punkcie s.

Po zakończeniu podziałów każda z obserwacji trafia do jednego z liści, a tam klasyfikacja odbywa się na zasadzie głosowania większościowego:

$$\hat{y}_m = \arg \max_{k \in \{0,1,2\}} \hat{p}_{mk} \tag{27}$$

gdzie:

- $\hat{y}_m$ – przewidywana klasa dla obserwacji wpadających do węzła m,
- $\hat{p}_{mk}$ – proporcja obserwacji należących do klasy k w węźle m.

Pojedyncze drzewa mają jednak pewną wadę. Cechują się niskim obciążeniem i bardzo wysoką wariancją. Przez to mają tendencję do overfittingu. Dlatego badacze skupili się na poszukiwaniu bardziej złożonych modeli. Jednym z takich rozwinięć były lasy losowe. Ich konstrukcja oparta jest na baggingu. Polega on na wygenerowaniu B próbek bootstrapowych Z^{*b}, gdzie do generowania pojedynczego podzbioru używamy losowania ze zwracaniem, które na każdym etapie przypisuje takie samo prawdopodobieństwo wylosowania dla każdego z elementów. Następnie dla każdej próbki konstruowane jest drzewo. Istnieją dwa główne podejścia dotyczące agregacji głosów w zadaniu klasyfikacji opartym na baggingu [11, s. 343]

Głosowanie większościowe (*hard voting*) [11, s. 343–345]:

$$\hat{G}_{\text{bag}}(x) = \arg \max_{k \in \{0,1,2\}} \sum_{b=1}^{B} I(\hat{C}_b(x) = k) \tag{28}$$

Aproksymacja prawdopodobieństw (*soft voting*) [9, s. 285]:

$$\hat{p}_k(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_k^{(b)}(x) \tag{29}$$

gdzie:

- B – całkowita liczba wygenerowanych drzew,
- $\hat{C}_b(x) \in \{0, 1, 2\}$ – klasa przewidziana przez b-te pojedyncze drzewo dla obserwacji x,
- $\hat{p}_k^{(b)}(x)$ – wyestymowane prawdopodobieństwo przynależności obserwacji x do klasy k przez b-te drzewo.

Jednak w Baggingu poszczególne drzewa są silnie ze sobą skorelowane i jeśli występuje cecha dominująca, to zostałaby użyta do podziału jako jedna z pierwszych, czyniąc drzewa podobnymi do siebie. Aby to poprawić, w lasach losowych przed dokonaniem podziału w każdym węźle konstruowany jest losowy podzbiór indeksów cech $M_{\text{sub}} \subset P$[7] o stałej liczebności $|M_{\text{sub}}| = m$ (gdzie zazwyczaj $m \approx \sqrt{p}$), który jest losowany bez zwracania [11, s. 346].

Dla zespołu B identycznie rozłożonych, lecz skorelowanych drzew wariancję definiuje wzór [9, s. 597]:

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^{B} \hat{T}_b(x)\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{30}$$

gdzie:

- $\hat{T}_b(x)$ – prognoza b-tego pojedynczego drzewa dla obserwacji x,
- ρ – korelacja parami pomiędzy wynikami poszczególnych drzew,
- σ^2 – wariancja pojedynczego drzewa.

Gdy liczba uśrednianych drzew rośnie do nieskończoności ($B \to \infty$), drugi człon równania $\left(\frac{1-\rho}{B}\sigma^2\right)$ zbiega do zera. Ostateczna wariancja zespołu pozostaje jednak ograniczona od dołu przez wartość $\rho\sigma^2$. Wybór podzbioru cech o określonej liczebności m podczas każdego podziału powoduje, że drzewa są budowane na mniej oczywistych partycjach, co wymusza zmniejszoną korelację drzew w lasach losowych względem baggingu, a więc zmniejszenie wartości współczynnika ρ.

W klasyfikacji trójklasowej predykcja modelu lasu losowego nie polega na głosowaniu większościowym, lecz jest ona średnią prawdopodobieństw zwróconych przez B drzew w lesie [9, s. 285]:

$$\hat{p}_k(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_k^{(b)}(x) \tag{31}$$

[7] P oznacza zbiór indeksów dostępnych zmiennych objaśniających: $P = \{1, 2, \ldots, p\}$.

3.1.3 Gradient boosting i model XGBoost

Boosting w odróżnieniu od baggingu polega na sekwencyjnym budowaniu modeli (w praktyce najczęściej drzew). Jest to metoda oparta na słabych predyktorach (ang. *weak learners*), czyli takich, dla których błąd klasyfikacji jest nieznacznie mniejszy od losowego zgadywania. Algorytm zakłada, że poprzez połączenie ze sobą kilku takich modeli otrzymana heurystyka znacząco zyska na jakości predykcyjnej [11, s. 347]. Ważnym elementem boostingu jest funkcja prognozująca [9, s. 341]:

$$f(x) = \sum_{m=1}^{M} \beta_m b(x; \gamma_m) \tag{32}$$

gdzie:

- $b(x; \gamma_m)$ – słaba funkcja bazowa parametryzowana przez wektor γ_m (reprezentujący zmienne podziału, progi i wartości w liściach). W dalszej części pracy będzie ona utożsamiana z drzewem decyzyjnym, które w praktyce bardzo często pełni tę rolę,
- β_m – współczynnik (waga) przypisany m-temu predyktorowi w zespole,
- M – całkowita liczba iteracji (słabych modeli).

Jednak wyznaczenie wszystkich parametrów $\{\beta_m, \gamma_m\}_{m=1}^{M}$ byłoby numerycznie niewykonalne. Dlatego stosuje się etapowe modelowanie addytywne, gdzie w każdym kolejnym kroku budowany jest nowy predyktor, który stara się poprawić błędy poprzedników.

$$(\beta_m, \gamma_m) = \arg\min_{\beta,\gamma} \sum_{i=1}^{N} L\Big(y_i, f_{m-1}(x_i) + \beta b(x_i; \gamma)\Big) \tag{33}$$

gdzie $L(\cdot, \cdot)$ to wybrana funkcja straty [9, s. 342].

Pierwszym skutecznym algorytmem boostingowym był AdaBoost. Jest on bezpośrednią realizacją powyższego schematu przy założeniu wykładniczej funkcji straty : [9, s. 343]:

$$L(y, f(x)) = \exp(-yf(x)) \tag{34}$$

gdzie $y \in \{-1, 1\}$ oznacza rzeczywistą klasę. Po podstawieniu tej wykładniczej funkcji straty do powyższego wzoru otrzymujemy:

$$\begin{aligned}
(\beta_m, \gamma_m) &= \arg\min_{\beta,\gamma} \sum_{i=1}^{N} \exp\Big(-y_i\big(f_{m-1}(x_i) + \beta b(x_i; \gamma)\big)\Big) \\
&= \arg\min_{\beta,\gamma} \sum_{i=1}^{N} \exp\big(-y_i f_{m-1}(x_i)\big) \cdot \exp\big(-y_i \beta b(x_i; \gamma)\big) \\
&= \arg\min_{\beta,\gamma} \sum_{i=1}^{N} w_i^{(m)} \exp\big(-\beta y_i b(x_i; \gamma)\big)
\end{aligned} \tag{35}$$

gdzie waga dla i-tej obserwacji zdefiniowana jest jako:

$$w_i^{(m)} := \exp\big(-y_i f_{m-1}(x_i)\big) \tag{36}$$

To podejście miało jednak zasadniczą wadę. Dla źle sklasyfikowanych obserwacji $y_i f_{m-1}(x_i) < 0$.

Wobec tego skrajny outlier, który został źle sklasyfikowany, całkowicie zdominuje powyższą sumę.

Dlatego poszukiwano dalej uogólnienia na dowolną różniczkowalną funkcję straty. Jednak wtedy wyznaczenie bezpośredniego wzoru byłoby analitycznie i numerycznie niemożliwe. Nowe podejście zostało zaproponowane w pracy J. Friedmana (2001)[9, s. 358–359]. Zasugerował on, żeby na zbiorze treningowym potraktować wartości prognozowane jako parametry funkcji straty. Wówczas aby zminimalizować globalnie funkcję straty $L(f) = \sum_{i=1}^{N} L(y_i, f(x_i))$ wykonuje się krok w kierunku ujemnego gradientu. Friedman zaproponował, aby dla każdej obserwacji wyliczyć pochodną cząstkową (gradient) względem prognozy z poprzedniej iteracji:

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x_i)=f_{m-1}(x_i)} \tag{37}$$

Kierunkiem najszybszego spadku jest ujemny gradient, którego składowe r_{im} Friedman w swojej pracy nazwał pseudoresztami.

$$r_{im} = -g_{im} \tag{38}$$

Po wyliczeniu wektora pseudoreszt nowe drzewo budowane jest poprzez minimalizację sumy kwadratów odchyleń od r_{im} [9, s. 359]:

$$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{N} (r_{im} - b(x_i; \gamma))^2 \tag{39}$$

Geometrycznie oznacza to rzut ujemnego gradientu z przestrzeni treningowej na reguły podziału drzewa. Po wyznaczeniu oryginalnych podziałów wartości w liściach nie są jeszcze optymalne z perspektywy oryginalnej funkcji straty. Dlatego, gdy podziały są już zablokowane, dla każdego liścia R_{jm} (gdzie j to indeks liścia) szukamy stałej wartości γ_{jm} [9, s. 359]:

$$\gamma_{jm} = \arg\min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma) \tag{40}$$

Na koniec aktualizowana jest wartość funkcji skumulowanej prognozy (gdzie J_m to liczba liści w m-tym drzewie).

$$f_m(x) = f_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm}) \tag{41}$$

– $\nu \in (0, 1]$ – parametr zwany współczynnikiem uczenia, który kontroluje wielkość kroku w kierunku ujemnego gradientu [9, s. 364]. Mniejsza wartość ν powoduje wolniejszą konwergencję, ale może prowadzić do lepszej generalizacji modelu.

Użyty w pracy XGBoost oparty na algorytmie Extreme Gradient Boosting stanowi rozwinięcie tej idei. Został on opracowany w 2016 roku przez badaczy T. Chena i C. Guestrina [4]. Opiera się on na rozwinięciu funkcji straty L w szereg Taylora do drugiego rzędu. Pozwala to na jednoczesną optymalizację struktury drzewa oraz wyznaczanie optymalnych wag w liściach w jednym kroku [4, s. 785–787]:

$$L^{(m)} \approx \sum_{i=1}^{N} \left[L(y_i, f_{m-1}(x_i)) + g_{im} b(x_i; \gamma_m) + \frac{1}{2} h_{im} b^2(x_i; \gamma_m) \right] + \Omega(b_m) \tag{42}$$

gdzie pochodne funkcji straty ewaluowane w punkcie $f(x_i) = f_{m-1}(x_i)$ definiuje się następująco:

– **Gradient** (pierwsza pochodna):

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x_i)=f_{m-1}(x_i)} \tag{43}$$

– **Hesjan** (druga pochodna):

$$h_{im} = \left[\frac{\partial^2 L(y_i, f(x_i))}{\partial f(x_i)^2}\right]_{f(x_i)=f_{m-1}(x_i)} \tag{44}$$

Natomiast $\Omega(b_m)$ stanowi człon regularyzacyjny (karę) [4, s. 785]:

$$\Omega(b_m) = \gamma J_m + \frac{1}{2}\lambda \sum_{j=1}^{J_m} w_{jm}^2 \tag{45}$$

gdzie:

– J_m - liczba liści

– w_{jm} – waga (predykcja) przypisana do j-tego liścia,

– γ oraz λ – hiperparametry kontrolujące siłę regularyzacji (odpowiednio dla liczby liści i wartości wag).

Zauważając, że człon $L(y_i, f_{m-1}(x_i))$ zależy tylko od poprzednich iteracji (czyli jest stały), możliwe jest jego pominięcie podczas zadania optymalizacji. Kolejno przegrupowując sumowanie z poziomu poszczególnych obserwacji na konkretne liście naszego drzewa z danej iteracji I_{jm} funkcja celu upraszcza się do postaci [4, s. 786]:

$$\tilde{L}^{(m)} = \sum_{j=1}^{J_m} \left[\left(\sum_{i \in I_{jm}} g_{im} \right) w_{jm} + \frac{1}{2} \left(\sum_{i \in I_{jm}} h_{im} + \lambda \right) w_{jm}^2 \right] + \gamma J_m \tag{46}$$

Minimalizacja powyższego problemu względem wag w_{jm} pozwala na wyznaczenie analityczne optymalnej postaci w każdym liściu, znacząco skracając czas pracy algorytmu [4, s. 786]:

$$w_{jm}^* = -\frac{\sum_{i \in I_{jm}} g_{im}}{\sum_{i \in I_{jm}} h_{im} + \lambda} \tag{47}$$

W kontekście klasyfikacji trójklasowej ($K = 3$), odpowiadającej trzem możliwym rezultatom meczu piłkarskiego (wygrana gospodarzy, remis, wygrana gości), algorytm XGBoost w każdej iteracji m buduje trzy osobne drzewa (po jednym dla każdej klasy $c \in \{0, 1, 2\}$). Wyjściem modelu dla i-tej obserwacji jest wektor surowych predykcji $[f_0(x_i), f_1(x_i), f_2(x_i)]^T$.

Prawdopodobieństwo przypisane klasie c wyznacza się za pomocą funkcji Softmax [9, s. 383]:

$$p_{ic} = \frac{e^{f_c(x_i)}}{\sum_{l=0}^{2} e^{f_l(x_i)}} \tag{48}$$

Funkcją straty dla pojedynczej obserwacji i jest ujemna log-wiarygodność (entropia krzyżowa):

$$\mathcal{L}_i = -\sum_{l=0}^{2} y_{il} \ln(p_{il}) \tag{49}$$

gdzie $y_{il} \in \{0, 1\}$ stanowi binarny wskaźnik rzeczywistego wyniku (kodowanie *one-hot*), przy czym $\sum_{l=0}^{2} y_{il} = 1$.

Aby wyznaczyć gradient g_{imc} oraz Hesjan h_{imc} względem predykcji modelu $f_c(x_i)$, należy obliczyć pochodne cząstkowe funkcji straty. W pierwszej kolejności wyznacza się pochodną prawdopodobieństwa p_{il} względem logitu $f_c(x_i)$, rozważając dwa przypadki:

1. Dla $l = c$:
$$\frac{\partial p_{ic}}{\partial f_c(x_i)} = \frac{e^{f_c(x_i)} \sum_{j=0}^{2} e^{f_j(x_i)} - e^{f_c(x_i)} e^{f_c(x_i)}}{\left(\sum_{j=0}^{2} e^{f_j(x_i)}\right)^2} = p_{ic}(1 - p_{ic}) \tag{50}$$

2. Dla $l \neq c$:
$$\frac{\partial p_{il}}{\partial f_c(x_i)} = \frac{0 - e^{f_l(x_i)} e^{f_c(x_i)}}{\left(\sum_{j=0}^{2} e^{f_j(x_i)}\right)^2} = -p_{il} p_{ic} \tag{51}$$

Korzystając z powyższych zależności oraz reguły łańcuchowej, gradient g_{imc} przyjmuje postać:

$$g_{imc} = \frac{\partial \mathcal{L}_i}{\partial f_c(x_i)} = -\sum_{l=0}^{2} y_{il} \frac{1}{p_{il}} \frac{\partial p_{il}}{\partial f_c(x_i)} \tag{52}$$

Rozbijając sumowanie na element dla $l = c$ oraz pozostałe elementy dla $l \neq c$, otrzymujemy:

$$g_{imc} = -y_{ic} \frac{1}{p_{ic}} \left(p_{ic}(1 - p_{ic})\right) - \sum_{l \neq c} y_{il} \frac{1}{p_{il}} \left(-p_{il} p_{ic}\right) \tag{53}$$

$$g_{imc} = -y_{ic}(1 - p_{ic}) + \sum_{l \neq c} y_{il} p_{ic} = -y_{ic} + y_{ic} p_{ic} + p_{ic} \sum_{l \neq c} y_{il} \tag{54}$$

Ponieważ $\sum_{l=0}^{2} y_{il} = 1$, powyższe równanie upraszcza się bezpośrednio do ostatecznej postaci błędu predykcji:

$$g_{imc} = p_{ic} - y_{ic} \tag{55}$$

W ścisłym sformułowaniu wieloklasowym, druga pochodna (Hesjan) jest macierzą symetryczną o wymiarach 3×3, której elementy pozadiagonalne (mieszane) wynoszą:

$$\frac{\partial^2 \mathcal{L}_i}{\partial f_c(x_i) \partial f_l(x_i)} = \frac{\partial g_{imc}}{\partial f_l(x_i)} = -p_{ic} p_{il} \quad \text{dla} \quad c \neq l \tag{56}$$

Ze względów optymalizacyjnych i obliczeniowych, XGBoost stosuje przybliżenie diagonalne Hesjanu [18]. Teoretyczny element leżący na głównej przekątnej macierzy Hesjanu, stanowiący drugą pochodną po tej samej klasie c, przybiera wówczas postać:

$$\frac{\partial^2 \mathcal{L}_i}{\partial f_c(x_i)^2} = p_{ic}(1 - p_{ic}) \tag{57}$$

Warto jednak zaznaczyć, że w rzeczywistej implementacji algorytmu XGBoost dla klasyfikacji

wieloklasowej stosowane jest tzw. **diagonalne górne ograniczenie Hesjanu** dane wzorem [18]:

$$h_{imc} = 2p_{ic}(1 - p_{ic}) \tag{58}$$

W konsekwencji, po podstawieniu wyznaczonych pochodnych cząstkowych (gradientu g_{imc} oraz diagonalnego Hesjanu h_{imc}) do ogólnego sformułowania, ostateczna, optymalna waga w^*_{jmc} przypisywana do j-tego liścia nowego drzewa dedykowanego klasie c przyjmuje postać:

$$w^*_{jmc} = -\frac{\sum_{i \in I_{jmc}} (p_{ic} - y_{ic})}{2 \sum_{i \in I_{jmc}} p_{ic}(1 - p_{ic}) + \lambda} = -\frac{\sum_{i \in I_{jmc}} g_{imc}}{\sum_{i \in I_{jmc}} h_{imc} + \lambda} \tag{59}$$

3.2 Modele Statystyczne

3.2.1 Model Poissona

Rozkład Poissona od dawna był wykorzystywany do predykcji wyników meczów piłkarskich. Już w 1982 roku M. J. Maher wykazał, że liczba bramek zdobytych przez zespół i w meczu t ma rozkład Poissona [13, s. 109–110]:

$$P(Y_{it} = y) = \frac{\lambda_{it}^y e^{-\lambda_{it}}}{y!} \tag{60}$$

gdzie $\lambda_{it} > 0$ to parametr intensywności (czyli oczekiwana liczba goli oraz wariancja: $E(Y_{it}) = \mathrm{Var}(Y_{it}) = \lambda_{it}$), natomiast $y \in \{0, 1, 2, \dots\}$.

Jako że $\lambda_{i,t}$ ma być koniecznie dodatnia, to w regresji poissonowskiej stosuje się logarytmiczną funkcję łączącą [11, s. 172–173]:

$$\log(\lambda_{it}) = x_{it}^T \beta \implies \lambda_{it} = e^{x_{it}^T \beta} \tag{61}$$

Opisane w rozdziale Przegląd Literatury parametry ataku danego zespołu i w meczu t: α_{it} oraz słabość obrony przeciwnika j w tym meczu: β_{jt} oryginalnie miały zmieniać się w czasie. Jednak dla dużych zbiorów danych (np. w tej pracy, gdzie wykorzystywany zbiór zawiera ponad 100 tysięcy meczów już po preprocessingu) problem stanowi wielowymiarowość, która destabilizuje skuteczną estymację parametrów. Jak wskazują Koopman i Lit [12, s. 167–169] optymalizacja wymagałaby stworzenia wektora o wymiarach $2J \times 1$, gdzie J oznacza ilość rozważanych drużyn. Ze względu na nieliniowość rozkładu Poissona estymacja nie mogłaby się odbywać za pomocą filtru Kalmana, lecz wymagałaby użycia bardziej zaawansowanych matematycznie metod np. `Importance Sampling`, które dla tak dużego zbioru danych byłyby nieoptymalne czasowo przez duży koszt obliczeniowy. Dlatego w pracy zdecydowano się dokonać redukcji wymiarowości:

$$\log(\alpha_i) + \log(\beta_j) \approx \beta_0 + \beta_1 Home + \beta_2 \mathrm{EloDiff} + \beta_3 \mathrm{EloTotal} + \beta_4 \mathrm{Form5Diff} + \gamma_{\mathrm{liga}} \tag{62}$$

gdzie β_3EloTotal reprezentuje sumaryczny poziom sportowy obu drużyn, a γ_{liga} to poprawka uwzględniająca specyfikę danej ligi. Omawiany model wykorzystuje z oryginalnej pracy Dixona i Colesa wyłącznie funkcję korygującą $\tau(x, y)$ dla niskich wyników bramkowych, bez parametrów ataku/obrony drużyn i wygaszania w czasie opisanych w podrozdziale 2.3.3.

Lewa strona tej równości oznacza tu sumę logarytmów [8] siły ataku oraz słabości obrony przeciwnika, co w klasycznym podejściu zaproponowanym w pracy [5] odpowiada ich iloczynowi $\alpha_i\beta_j$, który decyduje o liczbie oczekiwanych goli. Dla redukcji wymiarowości zostało to uzależnione od przewagi własnego boiska, poprawki ligowej oraz wyliczonych w preprocessingu cech formy i Elo, co pozwala na estymację jedynie kilku współczynników.

Warto uściślić, że w celu estymacji parametrów powyższego modelu, zbiór danych został przekształcony do tzw. formatu długiego, w którym każdy mecz jest reprezentowany przez dwa osobne wiersze (obserwacje), osobno dla zespołu gospodarzy oraz gości. Zmienna `Home` przyjmuje wówczas wartość 1 dla wiersza opisującego gospodarzy i 0 dla wiersza gości. Z kolei zmienna różnicowa `EloDiff` powstaje jako różnica rankingu Elo drużyny opisywanej w danym wierszu i jej przeciwnika (`TeamElo - OpponentElo`). Oznacza to, że dla tej samej pary drużyn w danym meczu, cecha ta zmienia swój znak na przeciwny w zależności od perspektywy (dla gospodarzy wynosi HomeElo − AwayElo, natomiast dla gości AwayElo − HomeElo). W ten sposób, na podstawie jednego oszacowanego wektora parametrów, intensywność zdobywania bramek dla gospodarzy (λ) oraz gości (μ) wyraża się odpowiednio jako:

$$\ln(\lambda) = \beta_0 + \beta_1 \cdot 1 + \beta_2(\text{HomeElo} - \text{AwayElo}) + \beta_3 \text{EloTotal} + \beta_4(\text{Form5Home} - \text{Form5Away}) + \gamma_{\text{liga}} \tag{63}$$

$$\ln(\mu) = \beta_0 + \beta_1 \cdot 0 + \beta_2(\text{AwayElo} - \text{HomeElo}) + \beta_3 \text{EloTotal} + \beta_4(\text{Form5Away} - \text{Form5Home}) + \gamma_{\text{liga}} \tag{64}$$

Do korekty niskich wyników zdecydowano się użyć podejścia zaproponowanego w pracy Dixona i Colesa [5, s. 268–270]. Polega ono na wprowadzeniu mnożnika $\tau(x, y)$ do niezależnego rozkładu brzegowego Poissona:

$$P(X_t = x, Y_t = y) = \tau(x, y) \cdot \frac{\lambda_t^x e^{-\lambda_t}}{x!} \cdot \frac{\mu_t^y e^{-\mu_t}}{y!} \tag{65}$$

gdzie μ_t oznacza oczekiwaną liczbę goli zdobytych przez drużynę przeciwną, a korekta jest opisana wzorem:

$$\tau(x, y) = \begin{cases} 1 - \lambda_t \mu_t \rho & \text{dla } x = 0, y = 0 \\ 1 + \lambda_t \rho & \text{dla } x = 0, y = 1 \\ 1 + \mu_t \rho & \text{dla } x = 1, y = 0 \\ 1 - \rho & \text{dla } x = 1, y = 1 \\ 1 & \text{w pozostałych przypadkach} \end{cases} \tag{66}$$

gdzie ρ oznacza parametr zależności, który następnie wybierany jest podczas optymalizacji. Przy czym nałożone są na niego warunki:

$$\max\left(-\frac{1}{\lambda_t}, -\frac{1}{\mu_t}\right) \leq \rho \leq \min\left(\frac{1}{\lambda_t \mu_t}, 1\right) \tag{67}$$

Parametry modelu zostały wyestymowane przez uproszczoną estymację dwuetapową. Jest to modyfikacja metody zaproponowanej w pracy [5], gdzie postulowana była jednoczesna optymalizacja wszystkich parametrów. To podejście jest jednak uzasadnione. Z powodu bardzo dużego rozmiaru zbioru danych jednoczesna optymalizacja wszystkich parametrów byłaby bardzo kosz-

[8] Logarytm jest tu opisaną wyżej funkcją łączącą.

towna obliczeniowo i niestabilna numerycznie, przy czym wartość parametru ρ nie ma znacznego wpływu na siły drużyn i stanowi jedynie małą korektę dla najniższych wyników.

W pierwszym kroku przyjęta została hipoteza zerowa dotycząca braku korelacji ($\rho = 0$) i wówczas $\tau(x, y) = 1$, a funkcja log-wiarygodności przyjmuje postać:

$$\ell_1(\beta) = \sum_{k=1}^{N} [x_k \log(\lambda_k) - \lambda_k + y_k \log(\mu_k) - \mu_k] - \sum_{k=1}^{N} [\log(x_k!) + \log(y_k!)] \tag{68}$$

Po wyestymowaniu współczynników β obliczamy intensywności strzelonych goli:

$$\hat{\lambda}_k = e^{x_{ik}^T \hat{\beta}}, \quad \hat{\mu}_k = e^{x_{jk}^T \hat{\beta}} \tag{69}$$

Następnie ustalamy wartości tych parametrów i wyliczamy optymalną wartość parametru ρ z funkcji celu, która jest teraz funkcją jednej zmiennej $\ell_2(\rho)$. W celu usprawnienia działania algorytmu założono, że szukany parametr jest w zakresie $[-0.2, 0.2]$, stąd:

$$\hat{\rho} = \arg \max_{\rho \in [-0.2, 0.2]} \ell_2(\rho) \tag{70}$$

Na koniec transformujemy wyestymowane oczekiwane zdobycze bramkowe na prawdopodobieństwo danego rozstrzygnięcia meczu, jako [5, s. 272–273]:

$$\begin{aligned} \text{Wygrana gospodarzy } (P_H): \quad & P_H = \sum_{x=1}^{M} \sum_{y=0}^{x-1} P(X = x, Y = y) \\ \text{Remis } (P_D): \quad & P_D = \sum_{x=0}^{M} P(X = x, Y = x) \\ \text{Wygrana gości } (P_A): \quad & P_A = \sum_{y=1}^{M} \sum_{x=0}^{y-1} P(X = x, Y = y) \end{aligned} \tag{71}$$

gdzie M oznacza maksymalną liczbę bramek, dla której ucinana jest nieskończona suma rozkładu Poissona, przy czym w kodzie użyto wartości $M = 15$. Ponieważ odcięcie rozkładu sprawia, że suma prawdopodobieństw wszystkich trzech możliwych rozstrzygnięć jest minimalnie mniejsza od 1, ich ostateczne wartości są normalizowane.

3.2.2 Regresja Logistyczna w ujęciu statystycznym

Wielomianowa regresja logistyczna oraz podejście oparte na funkcji Softmax opisują w istocie ten sam model probabilistyczny, wykorzystując jedynie dwie równoważne parametryzacje. Podczas gdy w uczeniu maszynowym powszechnie stosuje się symetryczną funkcję Softmax, w klasycznym ujęciu statystycznym dąży się do pełnej identyfikowalności modelu [9, s. 119]. Cel ten osiąga się poprzez zastosowanie asymetrycznej parametryzacji z tzw. kategorią odniesienia (klasą bazową) [11, s. 144–145]. Jeśli w ogólnym sformułowaniu Softmax przyjmiemy warunek identyfikowalności polegający na uznaniu jednej z klas za referencyjną i wyzerowaniu jej wektora parametrów (np. $\beta_0 = \mathbf{0}$, co implikuje $e^{x_i^T \beta_0} = e^0 = 1$), to ogólny wzór redukuje się do postaci z jawną klasą bazową w mianowniku.

Na potrzeby modelu wieloklasowej regresji logistycznej (`MNLogit`) zastosowano pomocnicze kodowanie wewnętrzne, w którym klasą referencyjną jest kod 0, reprezentujący wygraną gości.

Klasa 1 odpowiada remisowi, natomiast klasa 2 oznacza wygraną gospodarzy. Jest to wyłącznie techniczne przekształcenie wewnętrzne modelu stosowane na etapie wyznaczania współczynników. Wówczas logarytmy szans (ang. *log-odds*) dla pozostałych klas względem klasy referencyjnej definiuje się jako [9, s. 119]:

$$\begin{aligned} \log\left(\frac{\Pr(Y_i = 1 \mid x_i)}{\Pr(Y_i = 0 \mid x_i)}\right) &= \beta_1^T x_i \\ \log\left(\frac{\Pr(Y_i = 2 \mid x_i)}{\Pr(Y_i = 0 \mid x_i)}\right) &= \beta_2^T x_i \end{aligned} \tag{72}$$

gdzie β_1 i β_2 to dwa niezależne wektory.

Przekształcając podany układ, można otrzymać następujące wzory na prawdopodobieństwo warunkowe:

$$\begin{aligned} \Pr(Y_i = 1 \mid x_i) &= \frac{e^{\beta_1^T x_i}}{1 + e^{\beta_1^T x_i} + e^{\beta_2^T x_i}} \\ \Pr(Y_i = 2 \mid x_i) &= \frac{e^{\beta_2^T x_i}}{1 + e^{\beta_1^T x_i} + e^{\beta_2^T x_i}} \\ \Pr(Y_i = 0 \mid x_i) &= \frac{1}{1 + e^{\beta_1^T x_i} + e^{\beta_2^T x_i}} \end{aligned} \tag{73}$$

Co ważne, przed ostateczną ewaluacją modelu na zbiorze testowym oraz porównaniem jego wyników z innymi algorytmami, prognozowane prawdopodobieństwa wyjściowe są programistycznie przestawiane z powrotem do standardowego kodowania przyjętego globalnie w pracy, czyli: 0 = gospodarz, 1 = remis, 2 = goście. Zapobiega to jakimkolwiek rozbieżnościom metodologicznym przy obliczaniu metryk dokładności oraz funkcji straty.

Ponadto w modelu statystycznym nie uwzględniono kar L_1 i L_2, co pozwoliło na wyliczenie p-values dla poszczególnych cech. Estymacja parametrów odbywała się tu metodą największej wiarygodności, przy czym funkcja log-wiarygodności miała postać [9, s. 119–120]:

$$L(\theta) = \sum_{i=1}^{N} \left[y_{i1}(\beta_1^T x_i) + y_{i2}(\beta_2^T x_i) - \log\left(1 + e^{\beta_1^T x_i} + e^{\beta_2^T x_i}\right) \right] \tag{74}$$

gdzie y_{i1} oraz y_{i2} są binarnymi zmiennymi wskaźnikowymi przyjmującymi wartość 1, gdy rzeczywistym rezultatem i-tego meczu był odpowiednio remis ($Y_i = 1$) lub wygrana gospodarzy ($Y_i = 2$), a 0 w przeciwnym przypadku. Rzeczywisty wynik wygranej gości ($Y_i = 0$) jest tutaj reprezentowany przez sytuację, w której obie te zmienne przyjmują wartość 0 ($y_{i1} = y_{i2} = 0$), co eliminuje nadmiarowość parametrów modelu.

3.3 Metryki

Do oceny jakości użytych modeli zostały wykorzystane 3 metryki: `Accuracy`(Dokładność), `LogLoss` oraz `Brier Score`. Powołując się na książkę ISLP nie istnieje dokładnie jedna, właściwa metryka do oceny danego modelu [11, s. 27–28]. W modelowaniu meczów piłkarskich nie chodzi jedynie o mierzenie ilości wytypowanych poprawnie meczów, lecz także o oszacowanie pewności modelu i ryzyka pomyłki.

Pierwszą użytą metryką jest Accuracy [11, s. 35]:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} I(y_i = \hat{y}_i) \tag{75}$$

gdzie:

- y_i – rzeczywisty wynik i-tego meczu (0 – wygrana gospodarza, 1 – remis, 2 – wygrana gości),
- $\hat{y}_i$ – prognozowany wynik i-tego meczu,

Ma ona jednak pewną wadę, mierzy jedynie liczbę poprawnych predykcji modelu. Przez to nie nadaje się ona do oszacowania komercyjnej jakości modelu, ponieważ zgodnie z [6] najlepsza strategia gry oznacza obstawianie tylko tych meczów, które zakładają większą pewność naszego modelu względem tych użytych przez firmy bukmacherskie.

Kolejna rozpatrywana metryka to `LogLoss` [11, s. 405]:

$$\text{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=0}^{2} y_{ic}\log(p_{ic}) \tag{76}$$

gdzie:

- y_{ic} – binarny wskaźnik dla wyniku rzeczywistego,
- p_{ic} – prognozowane przez model prawdopodobieństwo, że i-ty mecz zakończy się wynikiem c.

W przeciwieństwie do `Accuracy`, `LogLoss` mierzy pewność modelu. Ocenia on, jak dobrze model dopasował swoje przewidywania względem faktycznego wyniku. Logarytmiczna kara sprawia, że dla największych pomyłek rośnie ona drastycznie szybko. Wymusza to na modelach odpowiednią kalibrację, co dobrze szacuje ich zdolności do użytku komercyjnego.

Ostatnią wykorzystaną metryką był `Brier Score` [2]:

$$\text{Brier Score} = \frac{1}{N}\sum_{i=1}^{N}\sum_{c=0}^{2}(p_{ic} - y_{ic})^2 \tag{77}$$

gdzie:

- y_{ic} – binarny indykator wyniku rzeczywistego,
- p_{ic} – prawdopodobieństwo przypisane przez model wynikowi c w i-tym meczu.

Jest to matematyczny odpowiednik błędu średniokwadratowego dla trójklasowej klasyfikacji. Maksymalna kara za pomyłkę w pojedynczym meczu wynosi 2, a minimalna to 0 dla predykcji idealnej. Wobec tego jest to bardziej stabilna numerycznie miara niż `LogLoss`, ponieważ jeden niespodziewany rezultat nie wpłynie w takim stopniu na ostateczną wartość metryki.

4 Dane i metodyka

4.1 Zbiór danych i jego charakterystyka

Zebrane dane pochodzą z otwartego repozytorium na GitHubie [8], które zawiera dane pobrane z ogólnodostępnego serwisu football-data.co.uk. [3] Zawierają one wyniki meczów piłkarskich oraz różne statystyki pochodzące z lig z całego świata. Wejściowo zbiór zawierał 230 557 wierszy i 48 kolumn zmiennych, opisujących mecze ligowe z lat 2000-2025. Ogromna skala danych pozwala, z jednej strony, na dobre wytrenowanie modelu, jednak ze względu na obecność lig niszowych oraz starych meczów stwarza problem wielowymiarowych braków danych, co może znacząco odbić się na jakości predykcyjnej oraz uwidocznić różnicę pomiędzy poszczególnymi ligami.

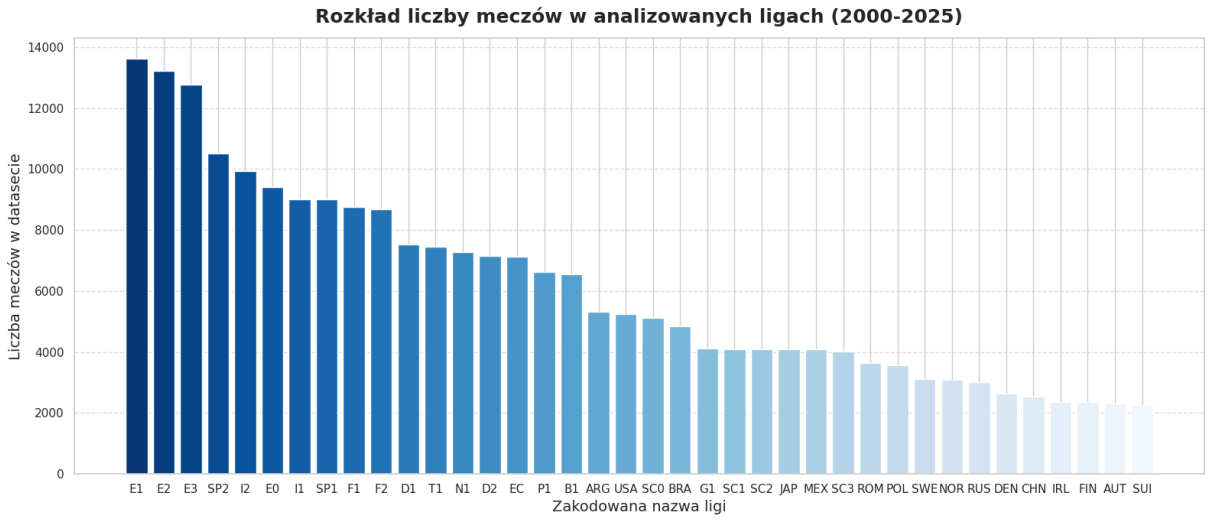
Zmienne w zbiorze danych można podzielić na kilka kategorii:

- Identyfikacyjne i czasowe: np. `Division` (zakodowana nazwa ligi), `MatchDate` (data meczu), `HomeTeam` (nazwa drużyny gospodarza), `AwayTeam` (nazwa drużyny gości),
- Sportowe: np. `Home/Away Elo` reprezentujący siłę drużyn gospodarzy i gości czy `Form3Home` reprezentujący formę na podstawie 3 ostatnich meczów domowych
- Statystyki pomeczowe: np. `FTHome` i `FTAway` reprezentujące liczbę goli zdobytych przez grające drużyny, czy też bardziej szczegółowe statystyki np. `Home/Away Shots` czy `Home/Away Yellow`. Zostały one jednak w większości usunięte w celu uniknięcia wycieku danych z przyszłości.
- Rynkowe: np. `OddHome`, `OddDraw`, `OddAway` reprezentujące kursy czyste typu 1X2, są też dostępne kursy na liczby bramek czy handicap. Możliwe jest dodanie ich do modelu w celu porównania wzrostu jakości względem obecnie stworzonych modeli.

Podział zbioru na cztery podgrupy pozwala na lepsze zrozumienie danych i stanowi naturalny punkt wyjścia do analizy eksploracyjnej.

4.2 Wstępna eksploracja danych

Wykorzystana baza danych charakteryzuje się szerokim przekrojem lig oraz zróżnicowanym poziomem rozgrywkowym 1. Najwięcej meczów pochodzi z lig angielskich, oznaczonych skrótami `E0`, `E1`, `E2`, `E3`, `EC`. Zauważalna jest też obecność lig niszowych, które będą wymagały dodatkowego preprocessingu ze względu na prawdopodobne braki danych.

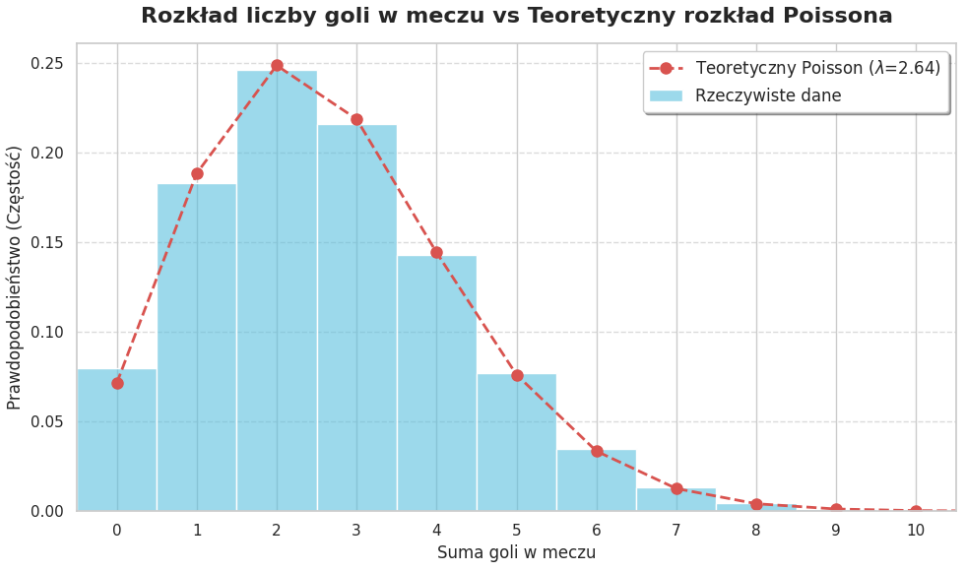


Rysunek 1. Rozkład liczby meczów w zależności od ligi

Legenda skrótów ligowych (kraj i poziom rozgrywkowy):

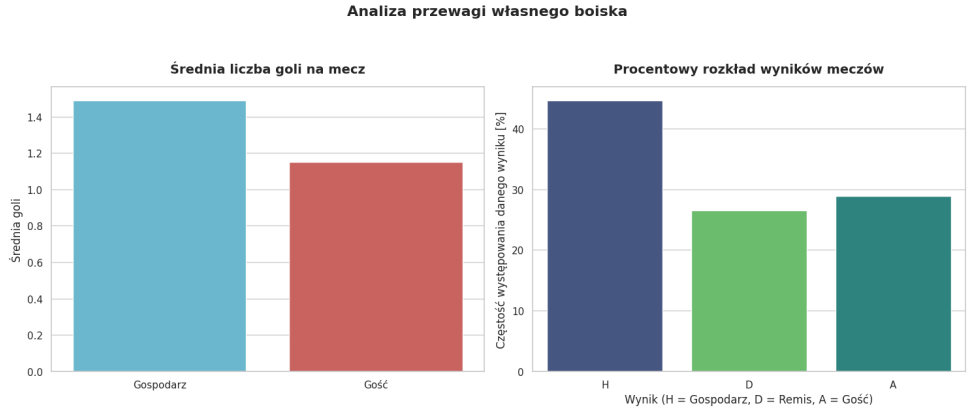
E0, E1, E2, E3, EC	Anglia (poziomy 1–5)	**SC0, SC1, SC2, SC3**	Szkocja (poziomy 1–4)
SP1, SP2	Hiszpania (1. i 2. liga)	**I1, I2**	Włochy (1. i 2. liga)
F1, F2	Francja (1. i 2. liga)	**D1, D2**	Niemcy (1. i 2. liga)
POL	Polska (Ekstraklasa)	**G1**	Grecja (Super League)
P1	Portugalia (Primeira Liga)	**N1**	Holandia (Eredivisie)
B1	Belgia (Pro League)	**T1**	Turcja (Süper Lig)
ROM	Rumunia (Liga I)	**USA**	USA (MLS)
ARG, BRA	Argentyna, Brazylia (1. liga)	**MEX**	Meksyk (Liga MX)
JAP, CHN	Japonia, Chiny (1. liga)	**DEN, RUS**	Dania, Rosja (1. liga)
SWE, NOR	Szwecja, Norwegia (1. liga)	**AUT, SUI**	Austria, Szwajcaria (1. liga)
FIN, IRL	Finlandia, Irlandia (1. liga)		

Rozkład liczby goli w poszczególnych meczach jest zbliżony do rozkładu Poissona, co jest zgodne z literaturą [13] (rysunek 2). Średnia liczba bramek wynosi 2,64 i stanowi parametr λ dla teoretycznej krzywej widocznej na rysunku.



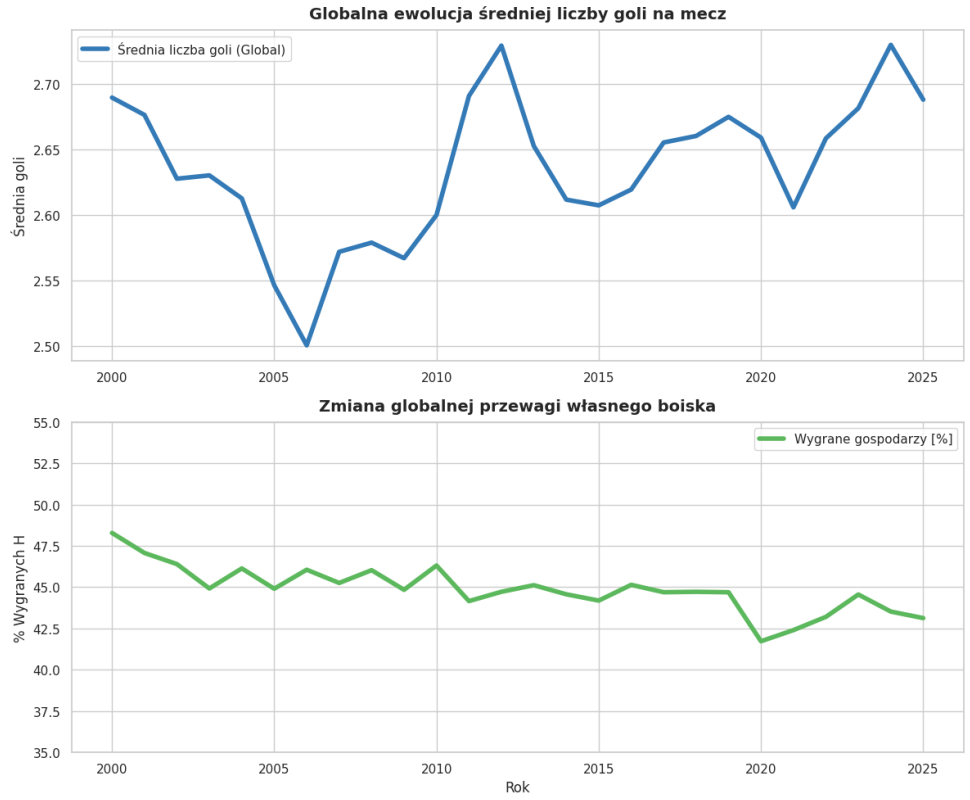
Rysunek 2. Porównanie rzeczywistego rozkładu liczby goli w meczu z teoretycznym rozkładem Poissona.

Kolejnym krokiem jest analiza przewagi własnego boiska. Na wykresie 3 widoczne jest, że drużyna gospodarza strzela średnio około 1,45 bramki na mecz podczas gdy goście tylko około 1,15. Ponadto procentowy rozkład wyników wynosi $45\%, 26\%, 29\%$ co uwypukla to zjawisko. Dlatego nasz model powinien koniecznie to uwzględniać.



Rysunek 3. Analiza przewagi własnego boiska

Średnia łączna liczba zdobywanych bramek przez drużyny w pojedynczym meczu oscyluje między 2,5 a 2,73 (rysunek 4). Jest ona stabilna i nie wykazuje żadnych tendencji do zmian w czasie w przeciwieństwie do przewagi własnego boiska, której wartość wykazuje ogólną tendencję spadkową, gdzie na przestrzeni 25 lat jest to spadek z 48% do 43% przy czym zauważalne jest drastyczne obniżenie w 2020 roku spowodowane prawdopodobnie pandemią COVID-19.



Rysunek 4. Globalna ewolucja lig

4.3 Preprocessing i czyszczenie danych

Surowy zbiór danych zawiera błędy, duplikaty i wycieki danych. Aby modele działały poprawnie kluczowy jest preprocessing i dobre przygotowanie danych, aby uczyły się faktycznych zależności zamiast szumu [11, s. 24].

4.3.1 Duplikaty i niszowe zespoły

Pierwszym krokiem wykonanym w preprocessingu było wyszukiwanie duplikatów. W celu ich usunięcia odfiltrowano powtarzające się trójki zmiennych jednocześnie: `MatchDate`, `HomeTeam`, `AwayTeam` (ostatecznie nie znaleziono żadnych duplikatów). Następnie usunięto rzadkie obserwacje, czyli drużyny z mniejszą liczbą meczów w zbiorze danych niż 10 (było 31 takich zespołów). Zbyt mała liczba obserwacji sprawiała, że estymacja siły tych drużyn byłaby najprawdopodobniej błędna i niestabilna [5, s. 272].

4.3.2 Eliminacja wycieku danych

Kolejno ze zbioru usunięte zostały wszystkie statystyki opisujące co działo się w trakcie meczów lub po ich zakończeniu.[9] [6] W ten sposób wyeliminowano błąd wycieku danych, który polega na tym, że model wie o statystykach, które są z przyszłości.[10] [11, s. 422–424, 461] Takie podejście fałszywie zawyżyłoby wyniki metryk na zbiorze walidacyjnym, w stosunku do rzeczywistej skuteczności na nowych danych. [9, s. 245–247]

4.3.3 Braki danych i selekcja lig

Przed przystąpieniem do modelowania wyników, konieczne było sprawdzenie ilości braków danych (rysunek 5), by zobaczyć, czy nie ma wśród nich jakiegoś wzorca. Okazało się, że kolumna `MatchTime` miała aż 131 469 luk, co stanowi ponad 50% obserwacji, przez co konieczne było jej usunięcie. Oprócz tego kolumny z Elo i kursami bukmacherskimi również miały znaczące ubytki na poziomie 70–80 tysięcy. [11, s. 515–516]

[9] Oczywiście poza wynikiem, który jest zmienną objaśnianą.

[10] Trzeba podchodzić do danych jak do szeregu czasowego, ponieważ mecze mają określony porządek chronologiczny. Można też wykorzystać te dane jako cechy okna czasowego ang. *lagged features*, które polegałyby na tym, że model wie tylko o cechach z przeszłości w pewnym odstępie czasowym od obecnego momentu.



Rysunek 5. Braki danych w surowym zbiorze

Następnie rozważono czy braki danych są rozłożone równomiernie po różnych ligach. Zauważono, że tak nie jest i część z nich ma ponad 50% ubytków w kolumnach z cechami Elo oraz bukmacherskimi (których ostateczne modele i tak nie wykorzystują jako cech wejściowych). Dlatego zdecydowano się je usunąć zostawiając w ramach wyjątku Ekstraklasę, aby zachować zgodność z tematem. Operacja ta miała na celu uniknięcie wprowadzania do modelu nadmiarowego szumu, który nie poprawiłby jakości predykcyjnej. [11, s. 515–516]

4.3.4 Przygotowanie cech do modeli liniowych i regularyzacji

W kolejnym kroku konieczne było przeskalowanie cech. Zaobserwowano, że zmienne objaśniające w bazie znacząco różnią się od siebie zakresem (rysunek 6). Dla przykładu zmienne dotyczące `Elo` mogą różnić się nawet o kilkaset jednostek, podczas gdy wskaźniki dotyczące `Formy` różnią się maksymalnie o kilka jednostek. Model regresji logistycznej czy funkcje kary są bardzo wrażliwe na taką dysproporcję; uniemożliwiłaby ona dobre skalibrowanie modelu, ponieważ model drastyczniej karałby współczynniki cech o mniejszych zakresach, dążąc do ich wyzerowania. [11, s. 241–242]

Nazwa cechy	Średnia	Odchylenie std. (std)	Zakres (Min - Max)	Opis cechy
HomeElo	1533,12	153,28	1103,13 - 2107,48	Ranking Elo gospodarza
AwayElo	1533,17	153,29	1103,13 - 2107,48	Ranking Elo gościa
Form5Home	6,72	3,26	0,00 - 15,00	Średnia punktów gospodarza (5 meczów)
Form5Away	6,93	3,27	0,00 - 15,00	Średnia punktów gościa (5 meczów)

Rysunek 6. Rozkłady kilku przykładowych cech numerycznych w surowym zbiorze są bardzo zróżnicowane, dlatego konieczne jest ich przeskalowanie.

4.4 Inżynieria cech

Surowe zmienne zawarte w zbiorze danych, stosowane bezpośrednio w swojej wyjściowej formie, nie niosą pełnej informacji na temat sił poszczególnych drużyn. Transformacja zmiennych poprzez uwzględnienie wzajemnych interakcji pomaga modelom dokładniej identyfikować nieliniowe zależności i zwiększa ich elastyczność [11, 9].

4.4.1 Cechy oparte na rankingu Elo

Wykorzystanie systemu Elo oraz jego matematycznych przekształceń w modelowaniu rezultatów sportowych jest podejściem silnie ugruntowanym empirycznie. Jak wykazali Hvattum i Arntzen [10], różnica punktów Elo między rywalizującymi zespołami stanowi jeden z najsilniejszych pojedynczych czynników predykcyjnych, pozwalających na uzyskanie stabilnych oszacowań prawdopodobieństw. W celu lepszego ukazania potencjalnego faworyta wprowadzono transformacje zmiennych dotyczących `Elo` drużyn grających przeciwko sobie:

$$\text{EloDifference} = \text{HomeElo} - \text{AwayElo}, \tag{78}$$

$$\text{EloAdvantage} = \frac{\text{EloDifference}}{\text{HomeElo} + \text{AwayElo}}. \tag{79}$$

4.4.2 Cechy oparte na zmiennych Formy

Następnie wprowadzono zmienne oparte na relacjach formy rywalizujących zespołów. Zmienna `FormMomentumDiff` reprezentuje dynamikę formy obu drużyn. Jeśli jej wartość jest dodatnia, oznacza to, że forma zespołu gospodarza z 3 ostatnich meczów jest lepsza niż ta z 5 w bardziej znaczącym stopniu niż ma to miejsce w przypadku zespołu gości.

$$\text{Form3Difference} = \text{Form3Home} - \text{Form3Away} \tag{80}$$

$$\text{Form5Difference} = \text{Form5Home} - \text{Form5Away} \tag{81}$$

$$\text{FormMomentumHome} = \text{Form3Home} - (\text{Form5Home} - \text{Form3Home}) = 2 \cdot \text{Form3Home} - \text{Form5Home} \tag{82}$$

$$\text{FormMomentumAway} = \text{Form3Away} - (\text{Form5Away} - \text{Form3Away}) = 2 \cdot \text{Form3Away} - \text{Form5Away} \tag{83}$$

$$\text{FormMomentumDiff} = (2 \cdot \text{Form3Home} - \text{Form5Home}) - (2 \cdot \text{Form3Away} - \text{Form5Away}) \tag{84}$$

4.5 Architektura potoku

Aby zautomatyzować cały proces przetwarzania danych i maksymalnie zawęzić jego objętość, zdecydowano się użyć architektury `Pipeline` z wykorzystaniem `ColumnTransformer` z biblioteki scikit-learn. Użycie powyższych narzędzi pozwala na zastosowanie dedykowanych transformacji w określonej kolejności [11, s. 275].

4.5.1 Przetwarzanie i kodowanie zmiennych (preprocessing)

Pierwszą wymaganą transformacją było kodowanie zmiennych kategorycznych. Podzielono je na dwie metody:

Kodowanie typu *one-hot* Użyte zostało tutaj klasyczne kodowanie przez `One-Hot Encoding` polegające na stworzeniu dodatkowych kolumn odpowiadających każdej z możliwych etykiet. [11, s. 82–86]

Kodowanie haszujące (`Hashing Encoding`) [14] W tym przypadku posłużono się `Hashing Encoder`. Było to spowodowane potencjalnym problemem wielowymiarowości, gdy liczba dodatkowych kolumn w klasycznym kodowaniu One-Hot byłaby bardzo duża [17]. `HashingEncoder` rozwiązuje ten problem, ponieważ używa funkcji haszującej do mapowania danych tak, by skompresować liczbę dodatkowych cech do pewnej ustalonej wartości (tutaj 32). Jednak niesie to za sobą pewien problem: obie drużyny są rzutowane do wspólnych 32 kubełków (ok. 20 zespołów na jeden), co przez kolizje i symetrię ogranicza wykorzystanie ich tożsamości w modelach.

Następnie wymagana była imputacja i standaryzacja zmiennych numerycznych, o czym napisano już we wcześniejszych podrozdziałach. Cechy Elo uzupełniono trzyetapowo: najpierw użyto metody `forward fill` polegającej na zastąpieniu brakujących danych ostatnią znaną wartością, w kolejnym kroku, jeśli dana drużyna nie ma meczów w bazie, to jej cechę Elo uzupełniono skumulowaną średnią historyczną danej ligi do momentu rozegrania meczu, a w przypadku absolutnych braków danych zdecydowano się wstawić bazową wartość tego rankingu – 1500. Zmienne formy zostały uzupełnione standardowo, poprzez `SimpleImputer` [9, s. 333]. Skalowanie odbyło się przy użyciu `StandardScaler`, który działa poprzez odjęcie średniej i podzielenie wyniku przez odchylenie standardowe [11, s. 241–242].

Tabela 1. Konfiguracja potoków wykorzystanych w procesie preprocessingu dla modeli uczenia maszynowego

Nazwa potoku	Zmienne wejściowe	Metoda imputacji	Transformacja / Kodowanie
`stats`	`HomeElo`, `AwayElo`, `Form3Home`, `Form3Away`, `Form5Home`, `Form5Away`	Mediana (`SimpleImputer`)	Standaryzacja (`StandardScaler`)
`date`	`DayOfWeek`, `Month`, `IsWeekend`	Najczęstsza wartość	Standaryzacja (`StandardScaler`)
`derived`	`EloDifference`, `FormMomentumDiff` itp.	Mediana (`SimpleImputer`)	Standaryzacja (`StandardScaler`)
`teams`	`HomeTeam`, `AwayTeam`	Stała (*'brak_danych'*)	`HashingEncoder` (32 cechy)
`league`	`Division`	Stała (*'brak_danych'*)	`OneHotEncoder`

4.5.2 Walidacja i podział danych

Dane z meczów piłkarskich nie mogą być traktowane jako zwykłe, niezależne rekordy w tabeli. Poszczególne wyniki drużyny są bardzo mocno zależne od poprzednich spotkań danego zespołu [11, s. 103]. Z tego względu klasyczna, losowa K-krotna walidacja krzyżowa (ang. *K-fold cross-validation*) jest tutaj metodologicznie niewłaściwa, ponieważ prowadziłaby do wycieku danych

z przyszłości do przeszłości [11, s. 176–177]. Aby rozwiązać powyższy problem, po wcześniejszym upewnieniu się, że mecze są uporządkowane chronologicznie, zdecydowano się wprowadzić parametr `shuffle=False`, który zapobiega wymieszaniu obserwacji w czasie. Istnieją także dedykowane metody walidacji dla szeregów czasowych jak chociażby walidacja przy użyciu okna kroczącego czy procedura *TimeSeriesSplit*. Ponadto dokonano dwukrotnego podziału:

Pierwszy podział: Ze względu na bardzo duży rozmiar danych zdecydowano się na odcięcie ostatnich 10% wyników na rzecz zbioru testowego. Zostanie on użyty tylko raz, wyłącznie do ostatecznej oceny jakości modeli.

Drugi podział: Aby móc dobrać najlepsze parametry modeli, zdecydowano się podzielić pierwsze 90% w stosunku 80:20 na zbiór treningowy i walidacyjny [9, s. 228]. Na drugim z nich przeprowadzono proces strojenia przy użyciu narzędzia `Optuna` [1]. Na koniec ponownie połączono oba podzbiory z drugiego podziału, aby przeprowadzić finalne uczenie przed ostateczną weryfikacją na zbiorze testowym.

4.5.3 Optymalizacja hiperparametrów

Aby maksymalnie zoptymalizować przeszukiwanie przestrzeni parametrów, zdecydowano się zrezygnować z klasycznych funkcji (np. `GridSearchCV`) [15], ponieważ są one nieefektywne i kosztowne obliczeniowo. Zamiast tego wykorzystano nowoczesny framework `Optuna` [1]. Polega on na zaawansowanym probabilistycznym podejściu bayesowskim. Zamiast losować parametry, analizuje rezultaty poprzednich prób w celu ulepszenia dotychczasowego najlepszego wyniku. Ponadto zmodyfikowana została funkcja celu, którą `Optuna` analizowała w 50 próbach (`trials`). Zamiast intuicyjnej maksymalizacji metryki `Accuracy` zdecydowano się na minimalizację `LogLoss`. Zmiana ta była kluczowa, ponieważ w predykcji meczów piłkarskich kluczowe jest wyestymowanie prawdopodobieństwa danego rezultatu, a `LogLoss` karze najmocniej pewność modelu, gdy jego predykcja jest błędna [11, s. 355–356]. Opisane procedury optymalizacyjne wiążą się ze sporym problemem. Dla tak dużego zbioru wielokrotny trening modeli na procesorze CPU byłby bardzo czasochłonny i znacząco obciążyłby używane urządzenie. Dlatego uznano, że lepszym rozwiązaniem będzie użycie akceleracji GPU, która jest dostępna na platformie `Kaggle`. Dla modeli regresji logistycznej oraz lasu losowego wykorzystano dedykowaną pod GPU bibliotekę NVIDIA `cuML` [16], natomiast XGBoost posiada wsparcie dla tej architektury [4].

Tabela 2. Wyniki optymalizacji hiperparametrów modeli uczenia maszynowego przy użyciu narzędzia Optuna [1] (na podstawie 50 prób dla każdego modelu)

Model ML i najmniejszy walidacyjny LogLoss	**Hiperparametr**	**Przedział przeszukiwań**	**Wartość optymalna**
	`max_depth`	$[3; 20]$	4
Drzewo decyzyjne	`min_samples_split`	$[10; 200]$	108
Val LogLoss: $1,0189$	`min_samples_leaf`	$[10; 100]$	62
	`criterion`	{*'gini'*, *'entropy'*}	*'gini'*
	`class_weight`	{*None*, `'balanced'`}	*None*
	`penalty`	{*'l1'*, *'l2'*, *'elasticnet'*}	`"l1"`
Regresja logistyczna	`C`	$[10^{-6}; 10^{4}]$	$0,01$
Val LogLoss: $1,0186$	`l1_ratio`	$[0,01; 0,99]$ gdy `penalty` = elasticnet	-
	`n_estimators`	$[200; 1500]$	581**
	`max_depth`	$[3; 8]$	4
	`learning_rate`	$[0,01; 0,2]$	$0,036$
	`subsample`	$[0,6; 1]$	$0,6602$
XGBoost	`colsample_bytree`	$[0,6; 1]$	$0,8634$
Val LogLoss: $1,0174$	`gamma`	$[0,00; 5,00]$	$1,29$
	`min_child_weight`	$[1; 10]$	9
	`reg_alpha`	$[10^{-4}; 10,0]$	$0,005445$
	`reg_lambda`	$[1,0; 10,0]$	$2,3926$
	`n_estimators`	$[100; 500]$	467
	`max_depth`	$[5; 18]$	5
	`max_features`	$[0,2; 0,8]$	$0,7102$
Las losowy	`max_samples`	$[0,5; 0,9]$	$0,5652$
Val LogLoss: $1,0177$	`min_samples_split`	$[2; 20]$	8
	`min_samples_leaf`	$[1; 10]$	8
	`n_bins`*	$[64; 256]$	131
	`split_criterion`*	$\{0; 1\}$	1
	`min_impurity_decrease`	$[0,00; 0,01]$	$0,0005$

* Parametry `n_bins` i `split_criterion` są specyficzne dla implementacji algorytmu lasu losowego w bibliotece `cuML`.

** Wartość 581 stanowi górny limit wylosowany przez framework Optuna, a nie rzeczywistą liczbę drzew w modelu końcowym, która została automatycznie ustalona za pomocą mechanizmu wczesnego zatrzymania (*early stopping* z cierpliwością 30 rund).

Na zbiorze walidacyjnym najlepszy wynik pod względem `LogLoss` osiągnął zaawansowany model `XGBoost`. Jednak różnice między poszczególnymi estymatorami są nieznaczne, co dowodzi, że nawet proste drzewo decyzyjne potrafi osiągać konkurencyjną jakość predykcji po dokładnej optymalizacji hiperparametrów. Po zakończeniu strojenia parametrów końcowe modele zostały dotrenowane na połączonym zbiorze treningowym i walidacyjnym.

5 Wyniki

5.1 Modele odniesienia

Aby sprawdzić poprawność implementacji modeli, przeprowadzono testy porównawcze z modelami odniesienia. W tym celu zbudowano 3 modele naiwne:

- Model losowy, który za każdym razem przypisuje równe szanse każdemu rozstrzygnięciu.
- Model częściowo losowy, który przypisuje prawdopodobieństwa na podstawie ogólnej historycznej proporcji wyników.
- Model zawsze gospodarz, który zawsze przypisuje wygraną gospodarzowi.

Ponadto dodano do zestawienia model bazowy oparty wyłącznie na zmiennych z dokładnymi wartościami Elo grających drużyn, a do estymacji prawdopodobieństw wykorzystano model regresji logistycznej.

Powyższe modele referencyjne osiągnęły następujące wyniki:

Tabela 3. Wyniki predykcji modeli referencyjnych na zbiorze testowym

Model bazowy	**Accuracy [%]**	**LogLoss**	**Brier Score**
Częściowo losowy (Prior)	44,17%	1,0730	0,6489
Model całkowicie losowy (33/33/33)	33,61%	1,0986	0,6667
Zawsze Gospodarz	44,17%	20,1217	1,1165
Model Elo (Home/Away Elo)	50,48%	1,0021	0,5990

Przedstawione modele naiwne pełnią jedynie funkcję porównawczą dla bardziej zaawansowanych algorytmów. Stanowią one dolną granicę użyteczności zadania predykcji. Aby miało ono realną wartość, każdy z zaawansowanych algorytmów musi osiągać korzystniejsze wyniki metryk niż te opisane w tej części. Natomiast model oparty na rankingu Elo ma pokazać siłę i znaczenie tego systemu w predykcji wyników meczów piłkarskich [10].

Szczególną uwagę zwraca bardzo wysoka wartość metryki `LogLoss` dla modelu „Zawsze Gospodarz", która wynosi aż $20,1217$. Wynika to bezpośrednio z konstrukcji tej metryki, która drastycznie karze pewne siebie, lecz błędne prognozy. Model ten przypisuje zawsze prawdopodobieństwo równe 1 zwycięstwu gospodarza oraz 0 remisowi i wygranej gości. W sytuacjach, gdy mecz kończy się innym rezultatem (remisem lub wygraną gości), model jest absolutnie pewny błędnego wyniku. Ponieważ logarytm z zera jest matematycznie nieokreślony ($\log(0) \to -\infty$), biblioteki uczenia maszynowego (np. [15]) automatycznie przycinają skrajne wartości prawdopodobieństwa do poziomu ustalonego epsilona. Generuje to olbrzymią karę punktową dla każdej takiej pomyłki, co po uśrednieniu na całym zbiorze testowym skutkuje anomalnie wysokim wynikiem końcowym metryki. Zjawisko to doskonale ilustruje wagę poprawnej kalibracji prawdopodobieństwa w modelach prognostycznych.

5.2 Porównanie skuteczności modeli uczenia maszynowego i statystycznych

5.2.1 Modele uczenia maszynowego

Pierwszą grupą modeli, które już zostały opisane w poprzednich rozdziałach były modele uczenia maszynowego. Po dostrojeniu parametrów zostały ostatecznie sprawdzone na zbiorze testowym.

Osiągnęły one następujące wyniki:

Tabela 4. Wyniki ewaluacji modeli uczenia maszynowego na zbiorze testowym

Model ML	**Accuracy [%]**	**LogLoss**	**Brier Score**
Decision Tree	50,35%	1,0016	0,5987
Logistic Regression	50,46%	1,0016	0,5987
Random Forest	50,45%	1,0006	0,5981
XGBoost	50,44%	1,0003	0,5979

Wyniki modeli uczenia maszynowego mogą nie wydawać się najwyższe. Od razu zauważalne jest, że podstawowy model oparty na rankingu Elo osiągnął większy wynik `Accuracy` niż najnowocześniejsze modele oparte na drzewach. Jednak najniższy błąd `LogLoss` osiągnął model `XGBoost`, a największe `Accuracy` miał model regresji logistycznej w pełnej wersji uczenia maszynowego. W kolejnej części przyjrzymy się dokładnie rozkładowi jakości prognoz i zobaczymy, dlaczego wartości metryk są obniżone.

5.2.2 Modele statystyczne

Kolejne były modele statystyczne, które również zostały opisane w poprzednich rozdziałach. Ze względu na użycie biblioteki `statsmodels` i metody największej wiarygodności możliwe było wyliczenie p-wartości dla poszczególnych cech. Ostateczne wyniki na zbiorze testowym wraz z wykazem istotnych zmiennych predykcyjnych przedstawiono w tabeli:

Tabela 5. Wyniki ewaluacji modeli statystycznych oraz istotność statystyczna zmiennych

Model statystyczny	**Accuracy [%]**	**LogLoss**	**Brier Score**	**Istotność statystyczna zmiennych objaśniających ($p < 0,05$)**
Regresja Poissona z korektą Dixona-Colesa	50,45%	1,0016	0,5987	`Home` ($\beta = 0,2840$; $z = 78,65$; CI: $[0,277; 0,291]$), `EloDiff` ($\beta = 0,0017$; $z = 105,99$; CI: $[0,00167; 0,00173]$), `EloTotal` ($\beta = 0,000091$; $z = 7,02$; CI: $[0,000066; 0,000115]$), `Form5Diff` ($\beta = 0,0032$; $z = 6,85$; CI: $[0,002; 0,004]$), Wybrane ligi (np. `SP2`, `SCO`, `I2` i inne)
Regresja Logistyczna (MNLogit)*	50,55%	1,0021	0,5990	**Równanie 1: Remis (Result = 1) vs Goście (Result = 0):** `EloDiff` ($\beta = 0,0032$; $z = 40,33$; CI: $[0,00304; 0,00336]$), Wybrane ligi (np. `F1`, `F2`, `I2`, `SP2`) **Równanie 2: Gospodarze (Result = 2) vs Goście (Result = 0):** `EloDiff` ($\beta = 0,0065$; $z = 86,45$; CI: $[0,006; 0,007]$), `Form3Diff` ($\beta = 0,0085$; $z = 2,23$; CI: $[0,001; 0,016]$), Wybrane ligi (np. `POL`, `SCO`, `G1`)

* Kategorią odniesienia (bazową) w modelu regresji logistycznej wieloklasowej (MNLogit) jest wygrana gości (`Result = 0`).

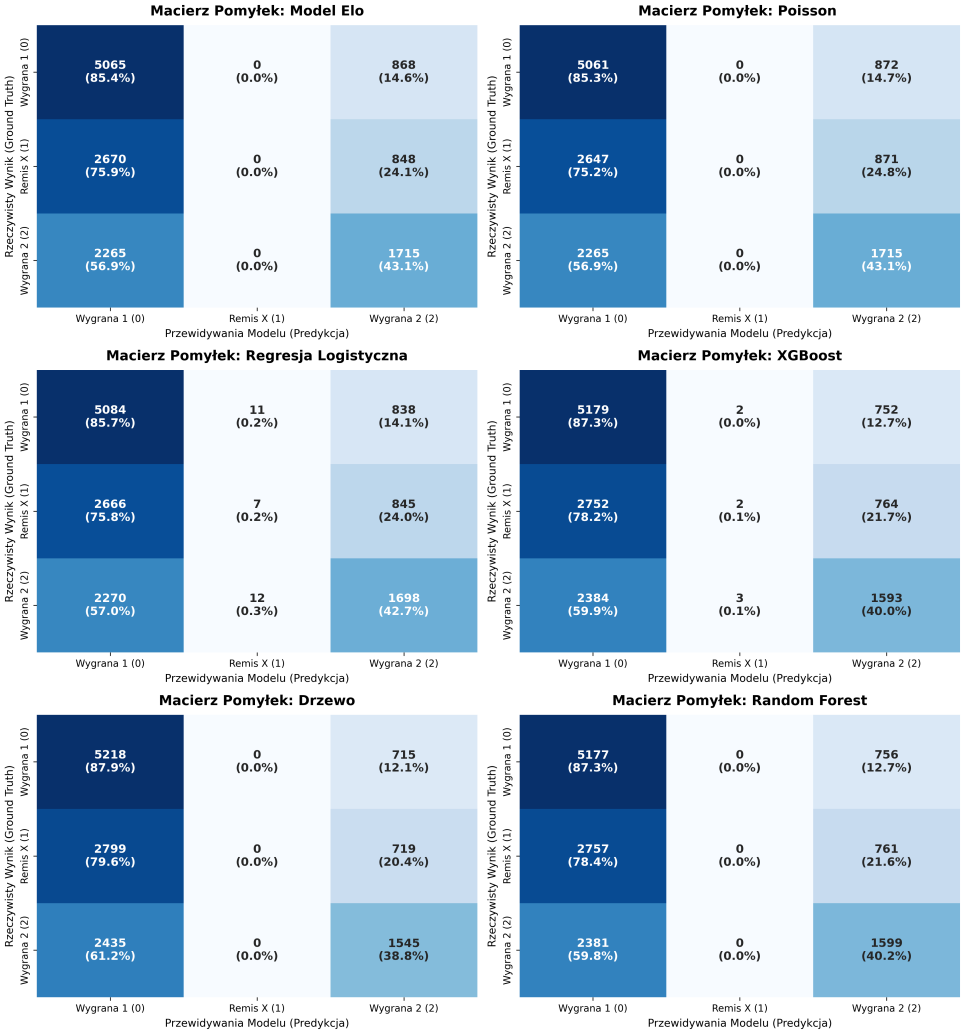
Osiągnęły one podobne wyniki metryk do modeli uczenia maszynowego, ustępując im nieznacznie pod względem funkcji straty logarytmicznej (`LogLoss`). Pokazuje to, że pomimo progresu technologicznego, klasyczne metody oparte na sprawdzonej matematyce dalej pozostają konkurencyjne. Aby móc skorzystać z biblioteki `statsmodels` konieczna była redukcja liczby zmiennych, co miało zapobiec błędom związanym z osobliwością macierzy. Wśród zmiennych objaśniających trzeba jednak wyróżnić `EloDiff`, która oprócz niskiej p-wartości cechuje się wysoką wartością statystyki z, co stanowi silny dowód na statystyczną istotność tej zmiennej i jej

niezerowy współczynnik w modelu [10] [11]. Warto również zauważyć, że dla modelu regresji Poissona została wyestymowana wartość parametru korygującego $\rho = -0,06$ (która w kodzie źródłowym została oznaczona jako ω). Ujemny znak wyestymowanego współczynnika jest w pełni zgodny z literaturą i pierwotnymi obserwacjami Dixona i Colesa, ponieważ delikatnie podbija prawdopodobieństwa dla najmniejszych wyników [5].

5.3 Zróżnicowanie skuteczności modeli

5.3.1 Analiza macierzy pomyłek

Choć globalne metryki potrafią dobrze oszacować ogólną jakość modeli, to nie dostarczają żadnej informacji na temat miejsc, w których modele popełniają błędy. Dlatego jednym z podstawowych narzędzi diagnostyki jest macierz pomyłek, pokazująca rozkład predykcji estymatorów względem prawdziwych wyników.



Rysunek 7. Macierze pomyłek modeli predykcyjnych

Wszystkie z badanych modeli najczęściej wskazywały wygraną gospodarza i to dla tego rozstrzygnięcia odnotowano najwięcej poprawnych predykcji. Jak wykazano w rozdziale *Dane i me-*

[11] Jej znaczenie predykcyjne zostało wykazane dalej, przy metodzie badania istotności cech `Permutation Feature Importance` dla modeli uczenia maszynowego.

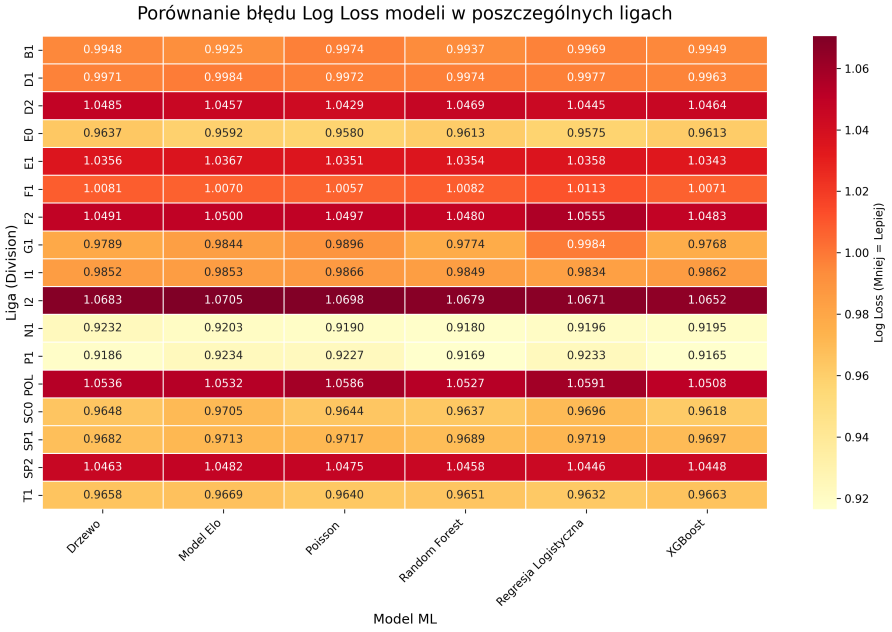
todyka, w dalszym ciągu silnie zaznacza się efekt przewagi własnego boiska, przez co najczęstszym rezultatem końcowym meczów jest wygrana gospodarzy. Następnym wyraźnym zjawiskiem jest niemal całkowity brak przewidywania remisów. Modele Elo, Poissona, Drzewa Decyzyjnego oraz Random Forest w ogóle nie generują takich prognoz na poziomie twardej klasyfikacji, natomiast XGBoost i regresja logistyczna robią to bardzo rzadko i najczęściej błędnie. Jest to bezpośredni efekt twardej reguły decyzyjnej `argmax` – szacowane przez modele prawdopodobieństwo remisu, choć czasami stosunkowo wysokie, prawie nigdy nie okazuje się najwyższą z trzech wartości w rozkładzie, przez co reguła decyzyjna konsekwentnie je odrzuca na rzecz wygranej któregoś z zespołów. Choć w przypadku regresji poissonowskiej (dzięki korekcie Dixona-Colesa) podbite zostały prawdopodobieństwa remisów przy niskiej liczbie bramek, to pierwotna różnica szans między wygraną faworyta a remisem była zbyt duża, aby ostateczna decyzja klasyfikatora uległa zmianie. Jednak w zastosowaniu komercyjnym zjawisko to nie pozostaje bez znaczenia. Pokazuje ono, że twarda klasyfikacja dyskretna jest na rynku zakładów bukmacherskich mało użyteczna. Dla podmiotów profesjonalnie zajmujących się prognozowaniem lub poszukiwaniem nieefektywności rynkowych kluczowa jest nie tyle wskazana przez regułę `argmax` pojedyncza klasa, ile precyzyjnie oszacowany i dobrze skalibrowany pełny rozkład prawdopodobieństwa. To właśnie te „miękkie" wartości probabilistyczne pozwalają na identyfikację tzw. zakładów wartościowych, w których model szacuje prawdopodobieństwo danego zdarzenia wyżej, niż wynikające z kursów oferowanych przez bukmacherów.

5.3.2 Porównanie jakości modeli w różnych ligach

Zauważono, że modele osiągnęły zbliżone wyniki w zakresie skuteczności predykcji. Jednak warto zwrócić uwagę na to, gdzie modele radzą sobie najlepiej a gdzie najgorzej. Pierwszym sposobem na sprawdzenie tego było rozważenie jakości predykcji w różnych ligach.

Tabela 6. Liczność zbioru testowego w podziale na poszczególne ligi piłkarskie

Skrót dywizji	Nazwa rozgrywek / Kraj	Liczba meczów
B1	Pro League (Belgia)	711
D1	1. Bundesliga (Niemcy)	746
D2	2. Bundesliga (Niemcy)	743
E0	Premier League (Anglia)	930
E1	Championship (Anglia)	1303
F1	Ligue 1 (Francja)	770
F2	Ligue 2 (Francja)	844
G1	Super League (Grecja)	156
I1	Serie A (Włochy)	929
I2	Serie B (Włochy)	909
N1	Eredivisie (Holandia)	737
P1	Primeira Liga (Portugalia)	745
POL	Ekstraklasa (Polska)	529
SC0	Premiership (Szkocja)	508
SP1	La Liga (Hiszpania)	939
SP2	Segunda División (Hiszpania)	1099
T1	Süper Lig (Turcja)	833
Suma całkowita (Zbiór testowy)		**13 431**



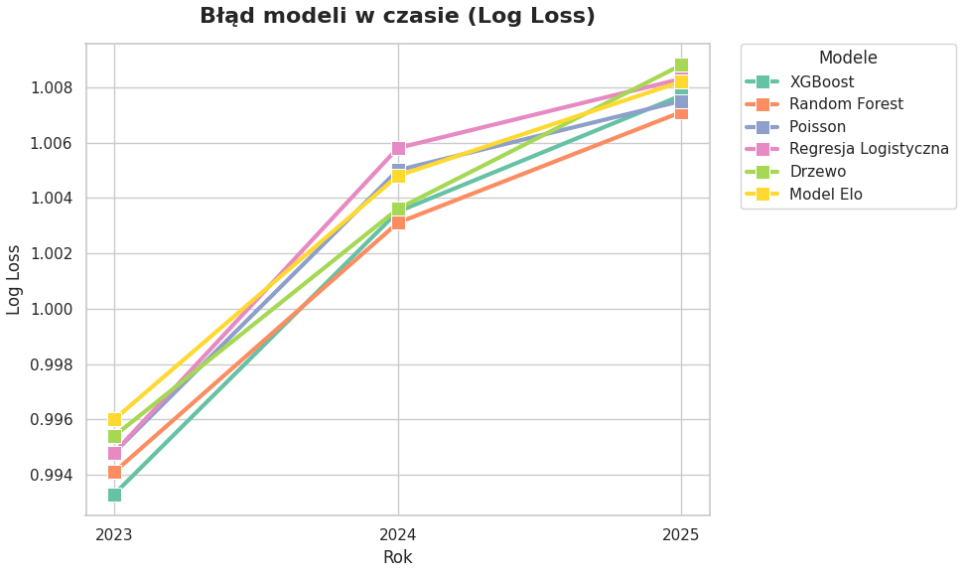
Rysunek 8. Porównanie błędu LogLoss w różnych ligach dla modeli uczenia maszynowego i statystycznych.

Od razu widoczna jest duża rozbieżność. W ligach takich jak holenderska czy portugalska `LogLoss` jest na poziomie około $0,918$ i `Accuracy` 57%, w najlepszych europejskich rozgrywkach (Hiszpania, Anglia, Niemcy) `LogLoss` wynosi około $0,96-0,97$ a `Accuracy` $53-54\%$. Natomiast w niższych ligach, jak np. drugi poziom we Włoszech, może on wzrosnąć aż do $1,065$ i `Accuracy`

42% dla najlepszych modeli. Różnice te mogą wynikać ze specyfiki samych rozgrywek: ligi charakteryzujące się najwyższą przewidywalnością zazwyczaj posiadają wyraźnych, dominujących faworytów oraz cechują się niższym udziałem wyników remisowych. Z kolei spadek jakości predykcji w niższych klasach rozgrywkowych stanowi obecnie jedynie hipotezę badawczą, która, choć wysoce prawdopodobna, nie została w pełni zweryfikowana empirycznie dla wszystkich analizowanych dywizji. Można przypuszczać, że zjawisko to wynika z gorszej jakości agregowanych danych: wyższego odsetka brakujących wartości, dużej rotacji zespołów na poziomie ligowym oraz ryzyka nieudokumentowanej imputacji braków już na poziomie tworzenia komercyjnych baz danych. Pośrednim, lecz namacalnym potwierdzeniem tej hipotezy jest przypadek polskiej Ekstraklasy, gdzie braki w kluczowych kolumnach z wartościami `Elo` sięgały aż 60%, co w sposób bezpośredni i zauważalny odbiło się na końcowych wartościach metryk.

5.3.3 Ewolucja błędu `LogLoss` w czasie

Następną zauważalną przyczyną gorszych wartości metryk jest przedział czasowy zbioru testowego. Pomimo że wykorzystano zaledwie 10% danych do testowania to i tak przekłada się na ponad 2 lata rozgrywek. Forma poszczególnych drużyn jest bardzo zmienna, stąd komercyjne modele są na bieżąco aktualizowane, aby zachować zdolność predykcyjną.

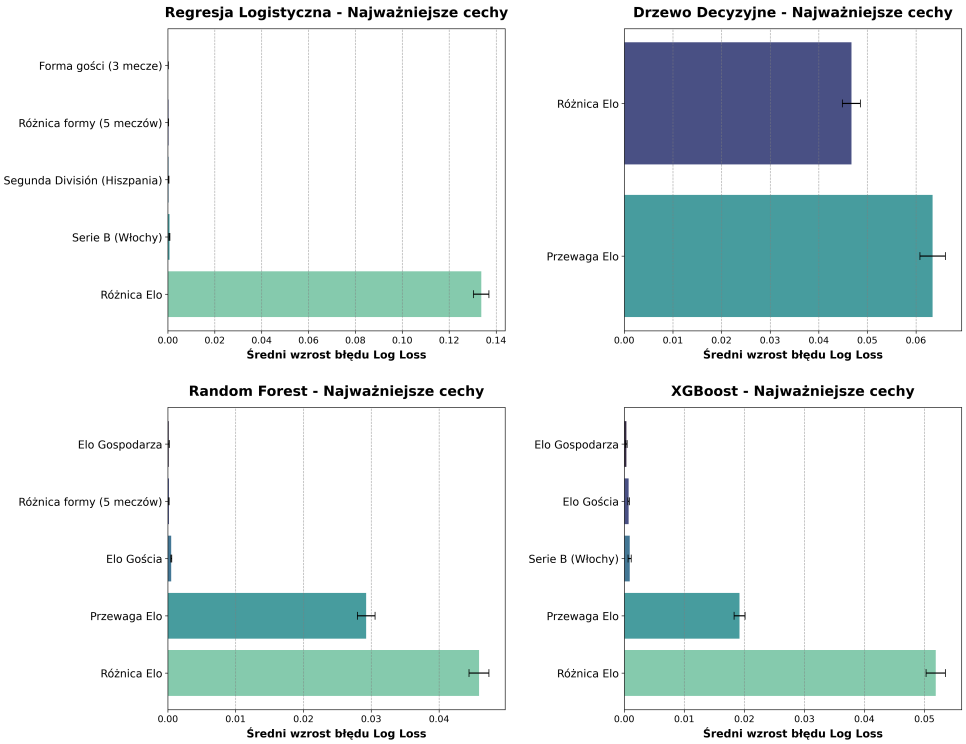


Rysunek 9. Ewolucja błędu LogLoss według roku w zbiorze testowym w latach 2023-2025

5.4 Analiza ważności cech

Jak wcześniej opisywano, w modelach ML nie ma możliwości wyliczenia p-wartości dla poszczególnych cech, przez co niemożliwe jest bezpośrednie wyświetlenie zmiennych, które miały największy wpływ na decyzję modelu. Aby rozwiązać ten problem, zdecydowano się użyć techniki `Permutation Feature Importance`. Polega ona na ponownym wymieszaniu wartości badanej cechy wśród wszystkich obserwacji w zbiorze testowym. W ten sposób zniszczony zostaje związek statystyczny między tą cechą a odpowiedzią modelu. Następnie ponownie wyliczana jest wartość metryki `LogLoss` w celu zidentyfikowania jego wzrostu. Im mocniej model zależał od danej cechy przy generowaniu predykcji, tym większy wzrost błędu odnotowywano po jej permutacji. Należy jednak pamiętać, że w przypadku silnie skorelowanych cech, technika ta może zaniżać

ich rzeczywistą wartość, ponieważ model może kompensować brak jednej zmiennej za pomocą jej skorelowanego odpowiednika [7]. Ponadto, aby uzyskane wyniki nie były dziełem przypadku, zastosowano tę procedurę 10-krotnie dla każdej cechy. Otrzymano następujące wyniki:



Rysunek 10. Permutation Feature Importance dla modeli ML

Na podstawie powyższego wykresu można zauważyć, że główny wpływ na predykcję ma ranking Elo oraz kilka zmiennych formy a pozostałe cechy to ich transformacje lub kodowanie drużyn i lig. Dla drzewa decyzyjnego, Random Forest dwoma najistotniejszymi cechami były `Przewaga Elo` oraz `Różnica Elo`. Warto dodać, że te cechy są niemal współliniowe, więc `Permutation Feature Importance` dzieli między nie jedną informację. Dla regresji logistycznej cecha `EloDifference` (będąca różnicą `HomeElo` i `AwayElo`) stanowi bezpośrednią kombinację liniową tych zmiennych. Ponieważ regresja logistyczna jest modelem liniowym, cecha ta nie dostarcza tutaj dodatkowej informacji, co widać po odniesieniu do tabeli 3. Ciekawą obserwacją jest obecność cechy `Serie B (Włochy)` dla XGBoost i regresji logistycznej, czyli ligi, w której modele miały najgorszą zdolność predykcyjną. Dzięki tej zmiennej estymatory otrzymują sygnał, że mają do czynienia z podzbiorem o wyższym stopniu losowości, co pozwala na spłaszczenie rozkładu prawdopodobieństwa i unikanie skrajnych decyzji. Znikomy wpływ cech kategorycznych powiązanych z tożsamością drużyn na ostateczne predykcje modeli wynika częściowo ze wspomnianego sposobu ich kodowania (wspólna przestrzeń haszująca).

Podsumowanie

Podsumowując, przewaga zaawansowanych modeli uczenia maszynowego ujawnia się głównie w metrykach probabilistycznych (`LogLoss` oraz `Brier Score`). Pod względem dokładności (`Accuracy`) wyniki są bardzo zbliżone, a dla modelu Elo oraz `MNLogit` wskaźnik ten jest nawet nieznacznie wyższy. Należy podkreślić, że różnice na poziomie $0,001 - 0,002$ `LogLoss` na zbiorze testowym (obejmującym 13 431 meczów) są na tyle małe, że ostateczny ranking modeli należy interpretować z dużą ostrożnością, gdyż różnice te mogą wynikać z losowego szumu. Ta niewielka skala poprawy wynika głównie ze specyfiki obecnego zbioru informacji (opartego głównie na historycznych ratingach Elo i podstawowej formie), w którym przy obecnym zestawie cech modele nieliniowe dają jedynie niewielką poprawę względem modeli liniowych. Nie oznacza to więc, że modele uczenia maszynowego nie są w stanie dać tutaj przewagi, lecz wskazuje na ograniczenie informacyjne zastosowanych cech.

Mimo upływu lat modele statystyczne nadal udowadniają, że potrafią być niezwykle konkurencyjne i ich skuteczność momentami przewyższała rezultaty nowoczesnych modeli uczenia maszynowego. Gdy problem badawczy jest dobrze zdefiniowany, a dane odpowiednio przygotowane, potrafią one osiągać stosunkowo dobre wyniki. Choć przyjęta metodyka w obu tych grupach różniła się, głównym celem było zachowanie matematycznego rygoru pracy oraz realnych zależności statystycznych. Z tego względu zrezygnowano z automatycznych ułatwień oferowanych przez nowoczesne architektury: przykładowo, algorytm XGBoost, zamiast wymagać wcześniejszej imputacji braków danych, mógł po prostu użyć swojej wbudowanej funkcji obsługi wartości pustych. Ponadto proces przygotowania i liczba cech dla modeli statystycznych były odmienne, ponieważ uzupełnienie wszystkich braków w złożony i automatyczny sposób oraz przetworzenie ich przez standardowy potok zaburzyłyby czyste wnioskowanie statystyczne i spowodowały problem współliniowości.

Następnie można zauważyć idee tworzonych od lat rankingów Elo. Testy statystyczne oraz technika `Permutation Feature Importance` pokazały, że odgrywają one znaczącą rolę w predykcji. Ponadto widać, jak istotne jest tworzenie cech pochodnych, ponieważ ułatwiają one modelom odnajdywanie zależności ukrytych w szumie informacyjnym.

Analizując macierze pomyłek, można łatwo potwierdzić tezę z badania Pope'a i Peela (1988) przywołaną w pracy Dixona i Pope'a [6]. Modele najczęściej nie wskazywały remisów jako ostatecznych przewidywań, a nieliczne próby ich wytypowania okazywały się błędne. Potwierdza to hipotezę, że remisy okazały się najtrudniejszą klasą do przewidywania przez zbudowane modele.

Choć surowe wyniki metryk nie były najlepsze, tak w rozbiciu na konkretne ligi można było zaobserwować znaczące dysproporcje. Uwypukla to, że twarda, ogólna klasyfikacja nie jest użyteczna dla gracza na rynku bukmacherskim. Ogólne wartości metryk `Accuracy` czy `LogLoss` są pogarszane przez ligi problematyczne, takie jak włoska Serie B, podczas gdy w rozgrywkach z wyraźną dominacją faworytów (np. Eredivisie), wartość metryki `Accuracy` okazała się aż o 15 punktów procentowych wyższa w stosunku do Serie B.

Dalszy rozwój pracy mógłby polegać na stworzeniu modeli hybrydowych, które łączą wysoką elastyczność predykcyjną modeli nowoczesnych, np. `XGBoost`, z tradycyjnym podejściem dotyczącym bramek na podstawie regresji Poissona, gdzie oczekiwana liczba goli byłaby dodatkową cechą. Kolejnym sposobem na ulepszenie podejścia jest wdrożenie zmiennych kroczącego okna czasowego, które na bieżąco aktualizowałyby statystyki z kilku ostatnich meczów. Mogłyby one znacząco podnieść jakość predykcyjną modeli, gdyż dyspozycja zespołów ma tendencję

do gwałtownych wahań. Możliwe jest także ulepszenie sposobu trenowania przez wprowadzenie metody rozszerzonego okna. Polega ona na trenowaniu modelu na historycznym podzbiorze danych, testowaniu go na kolejnym chronologicznie odcinku czasu, a następnie sukcesywnym włączaniu przetestowanego okresu do zbioru uczącego i przesuwaniu punktu prognozy w przód. Kolejnym kierunkiem rozwoju jest dodanie dodatkowych cech, które mogłyby wykazać znaczącą poprawę modeli zaawansowanych w stosunku do bazowych. Jeszcze innym ważnym kierunkiem rozwoju byłoby zastosowanie osobnego kodowania zmiennych kategorycznych dla gospodarzy i gości. Użycie niezależnych przestrzeni haszujących zapobiegłoby kolizjom oraz pozwoliłoby na asymetryczne uwzględnienie tożsamości zespołów i efektu własnego boiska.

Bibliografia

[1] Takuya Akiba i in. "Optuna: A Next-generation Hyperparameter Optimization Framework". W: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, s. 2623–2631. DOI: 10.1145/3292500.3330701.

[2] Glenn W. Brier. "Verification of forecasts expressed in terms of probability". W: *Monthly Weather Review* 78.1 (1950), s. 1–3.

[3] Joseph Buchdahl. *Football-Data.co.uk – Historical Football Results and Betting Odds Data*. https://www.football-data.co.uk. 2026.

[4] Tianqi Chen i Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System". W: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. ACM, 2016, s. 785–794. DOI: 10.1145/2939672.2939785.

[5] M. J. Dixon i S. G. Coles. "Modelling Association Football Scores and Inefficiencies in the Football Betting Market". W: *Journal of the Royal Statistical Society: Series C* 46.2 (1997), s. 265–280. URL: https://academic.oup.com/jrsssc/article/46/2/265/6990546.

[6] M. J. Dixon i P. F. Pope. "The value of statistical forecasts in the UK association football betting market". W: *International Journal of Forecasting* 20.4 (2004), s. 697–711. URL: https://www.sciencedirect.com/science/article/abs/pii/S016920700400010X.

[7] Aaron Fisher, Cynthia Rudin i Francesca Dominici. "All Models are Wrong, but Many are Useful: Learning a Variable's Importance for General Prediction and Least Squares". W: *Journal of Machine Learning Research* 20.177 (2019), s. 1–81. URL: http://jmlr.org/papers/v20/18-460.html.

[8] Adam Gábor. *Club Football Match Data (2000 – 2025)*. https://github.com/xgabora/Club-Football-Match-Data-2000-2025/. Faculty of Informatics and Information Technologies, Slovak University of Technology in Bratislava. 2025.

[9] Trevor Hastie, Robert Tibshirani i Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2009. URL: https://hastie.su.domains/ElemStatLearn.

[10] Lars Magnus Hvattum i Halvard Arntzen. "Using Elo ratings for predicting association football matches". W: *International Journal of Forecasting* 26.3 (2010), s. 460–470. DOI: 10.1016/j.ijforecast.2009.10.002.

[11] Gareth James i in. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2021. URL: https://www.statlearning.com.

[12] Siem Jan Koopman i Rutger Lit. "A Dynamic Bivariate Poisson Model for Analysing and Forecasting Match Results in the English Premier League". W: *Journal of the Royal Statistical Society: Series A* 178.1 (2015), s. 167–186. URL: https://academic.oup.com/jrsssa/article/178/1/167/7058470.

[13] M. J. Maher. "Modelling Association Football Scores". W: *Statistica Neerlandica* 36.3 (1982), s. 109–118. URL: https://onlinelibrary.wiley.com/doi/10.1111/j.1467-9574.1982.tb00782.x.

[14] Will McGinnis i in. *Category Encoders: a Python library of scikit-learn compatible categorical variable encoders*. https://contrib.scikit-learn.org/category_encoders. 2020.

[15] Fabian Pedregosa i in. “Scikit-learn: Machine Learning in Python”. W: *Journal of Machine Learning Research* 12 (2011), s. 2825–2830.

[16] RAPIDS Development Team. *cuML – GPU Accelerated Machine Learning*. https://github.com/rapidsai/cuml. 2026.

[17] Kilian Weinberger i in. “Feature hashing for large scale multitask learning”. W: *Proceedings of the 26th Annual International Conference on Machine Learning*. 2009, s. 1113–1120. DOI: 10.1145/1553374.1553516.

[18] XGBoost Contributors. *XGBoost Source Code – Multiclass Classification Objective*. https://github.com/dmlc/xgboost. 2026.

Spis załączników

1. Załącznik 1: Plik z kodem Python oraz baza danych konieczna do jego uruchomienia

Załączniki

Pełny kod źródłowy projektu, umożliwiający odtworzenie całego procesu przetwarzania danych oraz trenowania modeli, został dołączony do niniejszej pracy jako osobne archiwum elektroniczne w systemie APD.

Struktura i zawartość archiwum

W skład dołączonego pakietu wchodzą następujące elementy:

- `kod_licencjat_Stanislaw_Pawlak.ipynb` – główny notatnik Jupyter (*Jupyter Notebook*) zawierający pełen potok obliczeniowy: ładowanie danych, inżynierię cech, optymalizację hiperparametrów oraz właściwy trening modeli.
- `Matches.csv` – plik zawierający dane z meczów z lat 2000–2025.

Zalecenia dotyczące uruchomienia

W celu odtworzenia i ponownego uruchomienia obliczeń zaleca się skorzystanie ze środowiska chmurowego jak np. Kaggle Notebook wykorzystany w niniejszej pracy czy Google Colab. Platforma Kaggle jest szczególnie rekomendowana ze względu na długi dostępny czas treningu modeli przy użyciu akceleracji GPU, dużą pamięć operacyjną (RAM) sesji oraz bogaty zestaw preinstalowanych bibliotek.